POLYTECHNIC UNIVERSITY OF MADRID

HIGHER TECHNICAL SCHOOL OF TELECOMMUNICATIONS ENGINEERS

MASTER'S DEGREE IN ELECTRONIC SYSTEMS ENGINEERING

MASTER'S THESIS

DESIGN AND DEVELOPMENT OF A SOFTWARE EMULATION MODULE OF THE I^2C HARDWARE CONTROLLER OF THE STM32F4 MICROCONTROLLER FAMILY

ANTONIO LOTTI VILLAR

FEBRUARY 2026

MASTER'S DEGREE IN ELECTRONIC SYSTEMS ENGINEERING

MASTER'S THESIS

Título: DESIGN AND DEVELOPMENT OF A SOFTWARE EMULATION MODULE OF THE I^2C HARDWARE CONTROLLER OF THE STM32F4 MICROCONTROLLER FAMILY

Autor: ANTONIO LOTTI VILLAR

Supervisor: PEDRO JOSÉ MALAGÓN MARZO

Departamento: DEPARTMENT OF ELECTRONIC ENGINEERING

COMITÉ DE EVALUACIÓN

Presidente: .....

Vocal: .....

Secretario: .....

Los miembros del comité evaluador acuerdan otorgar la calificación: .....

Madrid, a ....., 2026

POLYTECHNIC UNIVERSITY OF MADRID

HIGHER TECHNICAL SCHOOL OF
TELECOMMUNICATIONS ENGINEERS

MASTER'S DEGREE IN ELECTRONIC SYSTEMS
ENGINEERING

MASTER'S THESIS

DESIGN AND DEVELOPMENT OF A SOFTWARE EMULATION MODULE OF THE I^{2}C HARDWARE CONTROLLER OF THE STM32F4 MICROCONTROLLER FAMILY

AUTHOR: ANTONIO LOTTI VILLAR

SUPERVISOR: PEDRO JOSÉ MALAGÓN MARZO

FEBRUARY 2026

Abstract

This project is part of the development of MICROLAB, a virtual laboratory for teaching embedded systems that uses QEMU as an emulation platform to replicate boards based on STM32F4 microcontrollers used at the UPM (Technical University of Madrid). The main objective was to incorporate the I^2C bus into the STM32F405 microcontroller and emulate one of the sensors used in laboratory exercises, the LIS3DH triaxial accelerometer, so that the firmware developed with STM32CubeIDE could run without modification on a completely virtual platform. To achieve this, the architectures of the STM32F405 and the LIS3DH, their register maps, and the interaction at the I^2C protocol level were analyzed in detail, defining a design that prioritizes register-level fidelity and compatibility with ST's HAL library.

On the microcontroller side, a complete model of the STM32F405 I^2C controller has been implemented in QEMU. This model is based on a finite state machine that replicates the START, address, read/write, and STOP sequences, as well as the generation of event and error interrupts (EV/ER) and the management of the CR1, CR2, SR1, SR2, and DR registers. On the sensor side, a detailed model of the LIS3DH as an I^2C slave device has been developed, implementing its 64-register map, the three operating modes (normal, high resolution, and low power), the configurable measurement ranges (± 2 g, ± 4 g, ± 8 g, ± 16 g), and the generation of the data-ready interrupt (DRDY) on the INT1 line. The model also exposes QOM properties (`accel-x/y/z`, `temp`) that allow for the controlled injection of acceleration and temperature data during testing, facilitating the systematic verification of the emulated device's behavior.

Both models have been integrated into a QEMU machine based on the Netduino Plus 2 board, which incorporates the STM32F405 SoC, peripheral address mapping, and correct routing of the NVIC and EXTI interrupt lines, connecting the LIS3DH as a slave on the I^2C1 bus at address `0x18`. Real firmwares developed with STM32CubeIDE and the HAL library were run on this platform, verifying I^2C bus initialization, accelerometer register configuration, auto-incrementing multi-byte readings, and the reception of data-ready interrupts. This demonstrates functional compatibility with real hardware in typical teaching scenarios. Overall, this work provides MICROLAB with a complete emulation of the STM32F405–LIS3DH I^2C

subsystem, enabling new virtual sensing and communication exercises without the need for physical hardware and establishing a reusable foundation for the future integration of other microcontrollers and peripherals from the STM32F4 family.

Keywords

QEMU, HAL, STM32, ARM, LIS3DH, emulation, virtualization, I^2C

Resumen

Este proyecto se enmarca en el desarrollo de MICROLAB, un laboratorio virtual para la docencia en sistemas empotrados que utiliza QEMU como plataforma de emulación para reproducir placas basadas en microcontroladores STM32F4 empleados en la UPM. El objetivo principal ha sido incorporar el bus I^2C en el microcontrolador STM32F405 y emular uno de los sensores utilizados en las prácticas de laboratorio, el acelerómetro triaxial LIS3DH, de forma que el firmware desarrollado con STM32CubeIDE pueda ejecutarse sin modificaciones sobre una plataforma completamente virtual. Para ello se han analizado en detalle las arquitecturas del STM32F405 y del LIS3DH, sus mapas de registros y la interacción a nivel de protocolo I^2C, definiendo un diseño que prioriza la fidelidad a nivel de registro y la compatibilidad con la librería HAL de ST.

En el lado del microcontrolador, se ha implementado en QEMU un modelo completo del controlador I^2C del STM32F405, basado en una máquina de estados finitos que reproduce las secuencias de START, dirección, lectura/escritura y STOP, así como la generación de interrupciones de evento y error y la gestión de los registros CR1, CR2, SR1, SR2 y DR. En el lado del sensor, se ha desarrollado un modelo detallado del LIS3DH como dispositivo esclavo I^2C , que implementa su mapa de 64 registros, los tres modos de operación (normal, alta resolución y bajo consumo), los rangos de medida configurables (± 2 g, ± 4 g, ± 8 g, ± 16 g) y la generación de la interrupción de datos disponibles (DRDY) en la línea INT1. El modelo expone además propiedades QOM (`accel-x/y/z`, `temp`) que permiten inyectar de forma controlada datos de aceleración y temperatura durante las pruebas, facilitando la verificación sistemática del comportamiento del dispositivo emulado.

Ambos modelos se han integrado en una máquina QEMU basada en la placa Netduino Plus 2, que incorpora el SoC STM32F405, el mapeo de direcciones de los periféricos y el encaminamiento correcto de las líneas de interrupción NVIC y EXTI, conectando el LIS3DH como esclavo en el bus I^2C1 con dirección `0x18`. Sobre esta plataforma se han ejecutado firmwares reales desarrollados con STM32CubeIDE y la librería HAL, verificando la inicialización del bus I^2C, la configuración de registros del acelerómetro, las lecturas multi-byte con auto-incremento y la recepción de interrupciones de datos listos, lo que demuestra la compatibilidad funcional con el hardware real en los escenarios típicos de las prácticas docentes. En con-

junto, el trabajo proporciona a MICROLAB una emulación completa del subsistema I^2C STM32F405–LIS3DH, habilitando nuevas prácticas virtuales de sensorización y comunicación sin necesidad de hardware físico y sentando una base reutilizable para la futura incorporación de otros microcontroladores y periféricos de la familia STM32F4.

Palabras clave

QEMU, HAL, STM32, ARM, LIS3DH, emulacion, virtualizacion, I^2C

Acknowledgments

This master's thesis wouldn't have been possible without the support of my family and friends.

Mom and Dad, thank you so much for always being there for me and supporting me in everything I've done.

Thanks to my friends, the monkeys, for their patience.

Thanks to my tutor, Pedro, for his guidance throughout the entire process.

Contents

List of Figures

List of Tables

List of Code

Glossary

This document is incomplete. The external file associated with the glossary 'main' (which should be called `Thesis.gls`) hasn't been created.

This has probably happened because there are no entries defined in this glossary. If you don't want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

This message will be removed once the problem has been fixed.

Chapter 1

Introduction

Invisible to many, but essential in modern society, microcontrollers, single-chip computers designed specifically for use in device control, communications, or process applications, have become fundamental elements in today's society. A single modern car may have as 25 differences microcontrollers that perform tasks ranging from audio system to global positioning systems, ignition control, engine control, and suspension balancing [3]. Many embedded systems use these devices as their core, and these in turn are becoming increasingly common and influential in progressively more critical sectors.Many embedded systems use these devices as their core, and these, in turn, are becoming increasingly common and influential in progressively more critical sectors. So much so that in 2024 the embedded systems market was valued at USD 110 billion [4]. Given this scenario, it is not difficult to imagine the sector's need for professionals trained in their design, operation, and implementation.

To train qualified professionals, having the necessary hardware is essential, something that is not always guaranteed, either in educational or professional settings. Often, both individuals and institutions face budgetary constraints when making decisions. In both educational and professional context, this translates into a lack of equipment for conducting real-world exercises with embedded systems. Many professionals agreed that one of the most challenging stages during project development is debugging and testing, since identifying and correcting programming errors is complex without access to the physical target hardware during development or the appropriate tools [5] [6].

In the educational sphere, the situation described in the previous paragraph is even worse. Students at public universities, which receive hundreds of new students each year, have very limited access to hardware. While the university itself usually provides the hardware to students, the sheer number of students makes it impractical for the university to provide one unit per student. This often forces students to conduct laboratory work in pairs and/or to share equipment among different groups. All of this limits students' opportunities to carry out independent experimentation.

Currently, software tools exist that allow the creation of virtual environments that emulate the behavior of real hardware, without the need for additional hardware. The software-based hardware emulation offered by these tools is one of the possible solutions to the problems mentioned above. By allowing software to be run and tested without physical hardware, debugging and behavior analysis are facilitated, and development time and costs are reduced. This is a clear advantage in both professional and educational settings, as it allows each individual—whether a student, professional, or anyone else with the goal of learning—to experiment individually with their own hardware, while simultaneously avoiding the typical costs and problems associated with real hardware, such as damaged circuit boards, wiring errors, or missing components.

The Polytechnic University of Madrid (UPM), as part of its Educational Innovation projects, is developing **MICROLAB** [7] and **SIVALSE** [8]. These two initiatives work together to address the educational problem described above. **SIVALSE** aims to create an open-source development environment with simulation and automatic validation tools to support challenge-based learning in electronic systems courses. **MICROLAB**, using the **SIVALSE** development environment, seeks to provide a virtual laboratory with a graphical interface for microcontroller-based electronic systems, allowing students to interact with the microcontroller and basic electronic components intuitively and in real time. All of this is achieved without the need for additional hardware, aside from the computer on which they are run. Throughout this thesis, artificial intelligence has been used as a tool for text translation and to ensure good writing style.

1.1 Objectives

During the academic year of the Master's in Electronic Systems, in the Embedded Systems course, several laboratory exercises were conducted using the STM32F4-DISCOVERY board. These exercises typically employed the I^2C communication protocol. The main objective of this thesis is to contribute to the development of the **MICROLAB** and **SIVALSE** projects by incorporating the I^2C bus into the STM32F4 microcontroller and accelerometer peripheral.

To achieve this main goal, the following **specific objectives** have been defined:

1. **Analyze and understand** the architecture of the STM32F405 microcontroller and the LIS3DH accelerometer, focusing on their communication through the Inter-Integrated Circuit (I^2C) protocol.

   This theoretical and technical understanding provides the foundation for accurate emulation of real hardware behavior.

2. **Develop an emulation model of the STM32F405 I^2C controller** within

the QEMU virtualization framework.

The model must implement register-level behavior, protocol timing, and interrupt generation to ensure compatibility with embedded firmware.

3. **Implement a virtual model of the LIS3DH accelerometer** as an I^2C slave device in QEMU.

   This model will reproduce the sensor's registers, operating modes, and three-axis acceleration data output, allowing the emulated STM32F405 to interact with it as if it were a physical device.

4. **Integrate both models into a unified virtual system** capable of executing real STM32 firmware within QEMU.

   The integration must simulate the interaction between the microcontroller and the accelerometer accurately, supporting complete I^2C data exchanges.

5. **Validate the emulated system using STM32 HAL**

   Firmware developed with STM32CubeIDE will be executed on the virtual platforms to confirm the functionality with the HAL.

6. **Document and evaluate** the system's performance, limitations, and educational potential within the MICROLAB environment.

   The results will serve as a reference for future integration of additional peripherals and tools to enhance virtual experimentation.

1.2 Overview

The remainder of the document is organized as follows:

- **Chapter 2** contextualizes the work and presents the tools used during development.
- **Chapter 3** analyzes the hardware to be emulated and the QEMU emulation tool.
- **Chapter 4** describes the system architecture design.
- **Chapter 5** describes the implementation and testing performed.
- **Chapter 6** presents the conclusions of the work and suggests possible areas for future improvement.

Chapter 2

Backgroung

This chapter aims to provide the necessary background to contextualize the work presented in this thesis, as well as to explain the tools used during its development.

2.1 Inter-Integrated Circuit (I^2C) Protocol

The Inter-Integrated Circuit protocol, often referred to as "I two C", but mostly known by its acronym I^2C, is widely known and used in a wide range of electronic applications where simplicity and low manufacturing cost are more important than speed. Developed in 1982 by Philips, now NXP Semiconductors, it was originally designed as "a two-wire bus system comprising a clock wire and a data wire for interconnecting a number of stations," addressing a practical need to control multiple integrated circuits in televisions and other consumer devices.

The first I2C-enabled integrated circuits also appeared in 1982 [2], operating in Standard-mode at 100kHz and using 7-bit addressing, which allowed up to 112 devices to share the same bus. At that time, I^2C remained proprietary to Philips and was not yet publicly standardized. In 1992, Philips released version 1.0 of the public I^2C specification, which enabled broader industrial adoption of the protocol.

The version 2.0 of the protocol was published in 1998. This version introduced the Fast-mode and increasing the maximum clock frequency from 100 kHz to 400 kHz, as well as adding 10-bit addressing to expand the address space from 128 to 1024 possible addresses. As embedded systems grew more complex and applications demanded higher data rates for inter-chip communication, the I^2C protocol continued to evolve without breaking backward compatibility, leading to the introduction of *Fast-mode Plus (Fm+)*, *High-speed mode (Hs-mode)*, and *Ultra Fast-mode (UFm)*, which increased the data rate to approximately 1 Mbit/s, 3.4 Mbit/s, and 5 Mbit/s, respectivel

I^2C Mode	Maximum Bit Rates
Standard-mode	100kbps
Fast-mode	400kbps
Fast-mode Plus	1Mbps
High-speed mode	3.4Mbps
Ultra-Fast mode	5Mbps

Table 2.1: Maximum Transmission Rates for Different I^2C Modes [1]

The I^2C protocol is widely used, to the point that it is currently a de facto global standard, implemented in over 1,000 different ICs manufactured by more than 50 companies. This is primarily due to its requirement of only two communication lines, making it a simple and inexpensive protocol for device manufacturers to implement.

2.1.1 I^2C Terminology

In the table 2.2 there are some important terms/concepts related with the I^2C protocol, that will be needed before entering in deep how the I^2C works.

TERM	DESCRIPTION
Transmitter	The device which sends data to the bus
Receiver	The device which receives data from the bus
Master	The device which initiates a transfer, generates clock signals and terminates a transfer
Slave	The device addressed by a master
Multi-master	More than one master can attempt to control the bus at the same time without corrupting the message
Arbitration	Procedure to ensure that, if more than one master simultaneously tries to control the bus, only one is allowed to do so and the winning message is not corrupted
Synchronization	Procedure to synchronize the clock signals of two or more devices

Table 2.2: Definition of I^2C-bus terminology [2]

2.1.2 I^2C Features & Overview

Like other serial communication protocols, the I^2C bus has a serial data line (SDA) and a serial clock line (SCL). However, unlike other serial protocols, the I^2C

protocol was created to allow communication between multiple devices over these two shared lines. The SCL line, mostly managed by the master device, serves to clock data in or out of the slave device. The other, the SDA line, sends or receives data to and from the slave devices. Figure 2.1 shows a standard I^2C physical layer implementation.

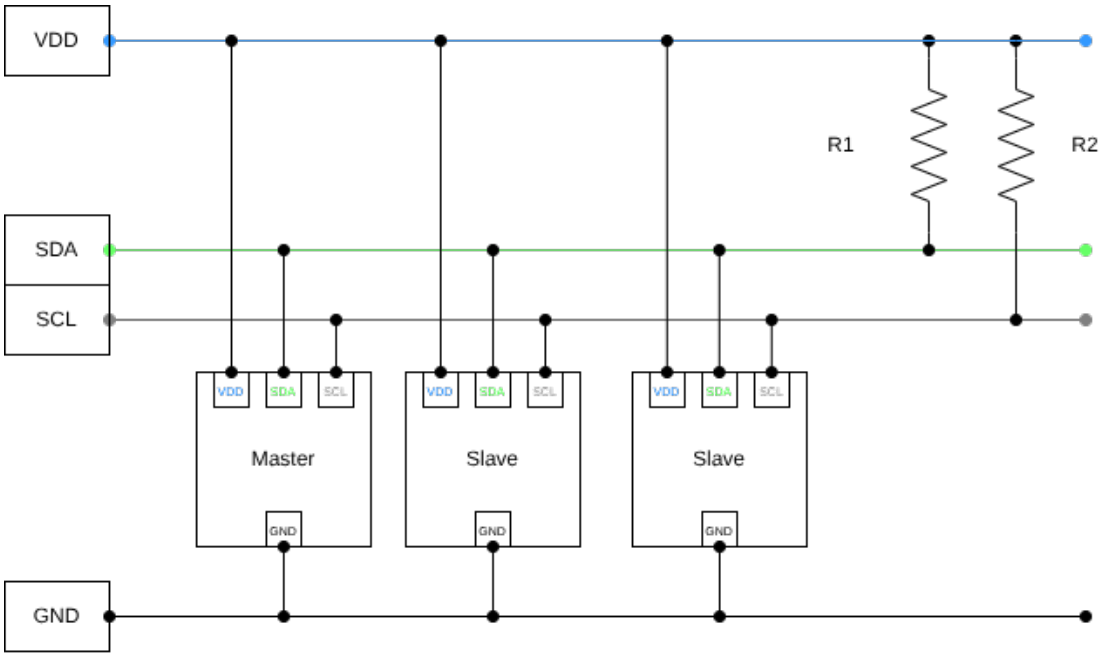


Figure 2.1: Typical I^2C implementation.

The I^2C protocol is designed so that the SDA and SCL lines have an open-drain connection with all devices on the bus. This requires a pull-up resistor connected to a common voltage source, as shown in Figure 2.1. The pull-up resistors define the high logic level and ensure that contention does not occur on the bus, where one device attempts to activate the line while another deactivates it. This prevents potential damage to the controllers.

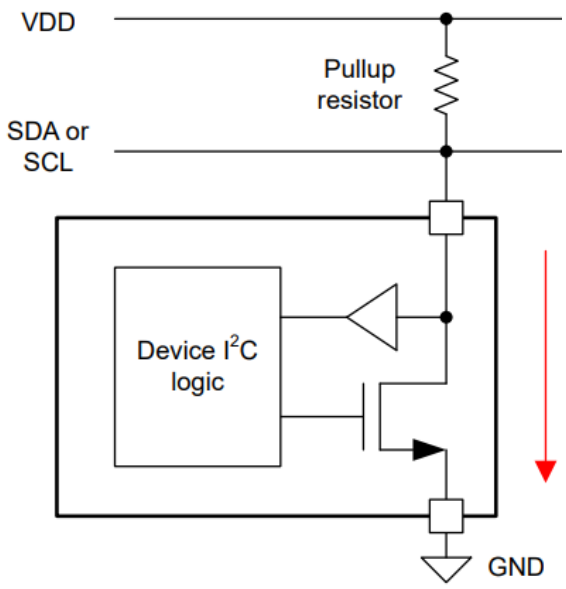


Figure 2.2: Open Drain.

The I^{2}C protocol is designed as a half-duplex communication scheme, in which only a single master or slave device transmits data on the bus at any given time. Each device connected to the bus is identified by a unique address and can operate as either a transmitter or a receiver. During any given transaction, a simple master-slave relationship is established: masters can act as master-transmitters or master-receivers, where the master is the device that initiates the data transfer and generates the clock signals required for communication.

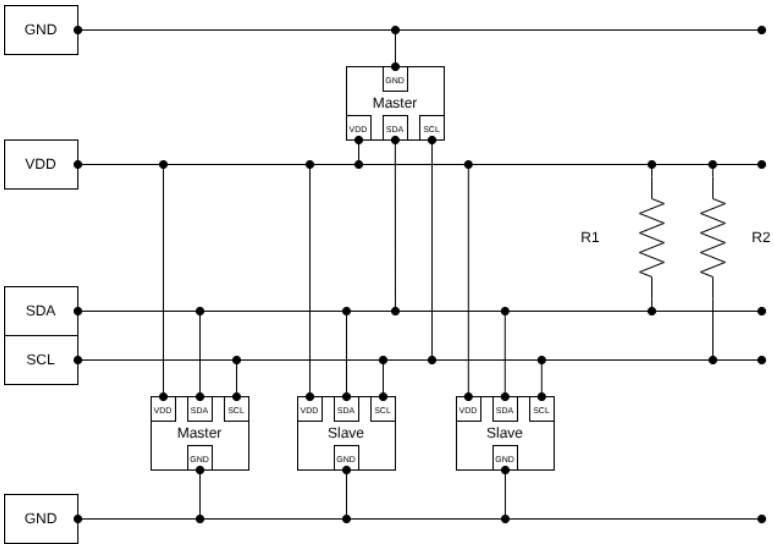


Figure 2.3: Multi-master example

There can be more than one master connected to the same bus, both capable of controlling it. Since masters are usually microcontrollers, an example of data transfer between two microcontrollers connected to the I^2C bus is shown in Figure 2.3. This example highlights the master-slave and transceiver roles that can arise on the I^2C bus. These roles are not permanent; they depend solely on which device initiates communication and the current direction of the data transfer.

A data transfer between two microcontrollers, A and B, proceeds as follows [2]:

1. When microcontroller A wants to send information to microcontroller B:
   - Microcontroller A (master) addresses microcontroller B (slave).
   - Microcontroller A (master-transmitter) sends data to microcontroller B (slave-receiver).
   - Microcontroller A terminates the transfer.
2. When microcontroller A wants to receive information from microcontroller B:
   - Microcontroller A (master) addresses microcontroller B (slave).
   - Microcontroller A (master-receiver) receives data from microcontroller B (slave-transmitter).
   - Microcontroller A again terminates the transfer.

No matter the situation, even when he is the one receiving the information, the master (the microcontroller A in the example) generates the clock signal and ends the transfer, always.

When multiple microcontrollers are connected to the I^2C bus, more than one master may attempt to initiate a transfer at the same time. So, in order to prevent conflicts, an arbitration procedure is used, relying on the wired-AND[2] connection of all I^2C interfaces to the bus. If two or more masters attempt to place data on the bus simultaneously, the first master that attempts to transmit a logical "1" while another master drives a logical "0" will lose arbitration and must cease transmitting. During arbitration, the clock signals on SCL become a synchronized combination of the clocks generated by all contending masters through the wired-AND connection to the clock line.

[1] Half-duplex communication is a transmission mode where data can be sent and received in both directions, but not simultaneously; only one party can transmit at a time, requiring the other to wait until the transmission is complete before responding.

[2] It is type of logic gate that implements the Boolean AND operation using passive components such as diodes and resistors, typically leveraging open collector or similar outputs

2.1.3 Protocol

This section of the chapter will explain how the protocol used in I^2C communication works to transfer data. In the I^2C protocol, data is transferred in messages, which are divided into two types of frames, an address frame, where the master specifies the destination device, and one or more data frames. The frames are 8-bit data units exchanged between the master and slave. Each message also includes START and STOP conditions, a read/write bit, and ACK/NACK bits between the data bytes to facilitate handshake and error signaling.

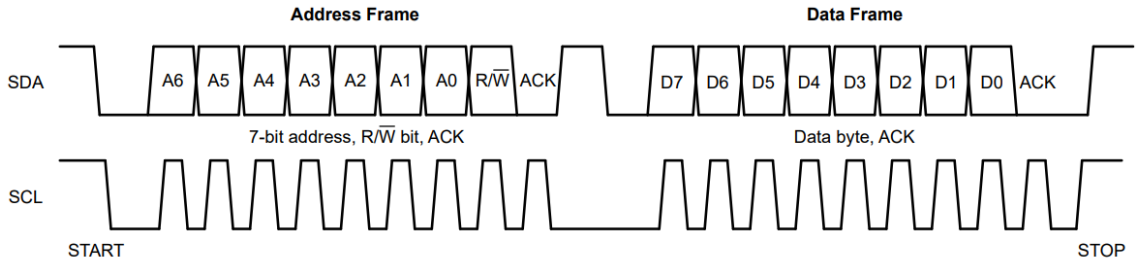


Figure 2.4: I2C Frame

Start & Stop Condition

Every bus transaction begins with a START condition (ST). A START condition is defined as a HIGH-to-LOW transition on SDA while SCL remains HIGH. Conversely, a STOP condition (SP) is defined as a LOW-to-HIGH transition on SDA while SCL remains HIGH. Both START and STOP conditions are always generated by the master.

Once a START condition occurs, the bus is considered busy; it becomes free again a specified time after a STOP condition. If the master issues a repeated START instead of a STOP, the bus remains busy, and the master can initiate a new transaction without releasing control of the bus.

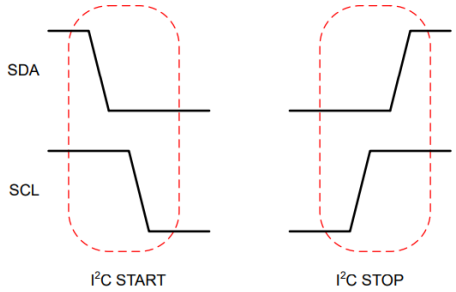


Figure 2.5: Start & Stop Conditions

Data To Transfer

Data on the SDA line is always transferred in bytes of 8-bits. The valid data bit must be stable while SCL is HIGH: data is placed on SDA while SCL is LOW and is sampled after SCL goes HIGH. As a result, SDA is only allowed to change state when SCL is LOW, except when generating START or STOP conditions. The number of data bytes in a transfer is not limited by the protocol itself, but each byte must be followed by a single acknowledge (ACK) bit.

Address Phase

After the master generates a START condition, it transmits the first byte on the bus. In the standard 7-bit scheme, the upper seven bits of this byte contain the slave address, and the least significant bit indicates the transfer direction: read or write. Every device on the bus compares these seven address bits with its own configured address; a device whose address matches considers itself selected and responds accordingly.

Acknowledge (ACK)

After every transmitted byte, there is an acknowledge phase. The ACK bit is used by the receiver, whereby the receiver can signal the transmitter that the byte was received (not the byte, rather the byte is all byte transfers, i.e. signal the transmitter that the byte transfer was completed). The master generates all clock pulses, including the ninth clock cycle, which is used for ACK/NACK. During this ninth clock, the transmitter releases SDA so the receiver can drive it.

An ACK is signaled when the receiver pulls SDA LOW and holds it while SCL is HIGH for that clock cycle. A NACK is signaled when SDA stays HIGH during the ninth clock cycle.

After a NACK, the controller usually either does a STOP to end the transfer or a repeated START to initiate a new transaction. NACKs can occur for various reasons, such as the absence of a device at the transmitted address, the device that was addressed is busy and cannot respond, the receiver is unable to process the command or data that was sent, the receiver has a busy status meaning it cannot accept additional data, or a controller-receiver pair uses NACK to signal a termination of the transfer to the target transmitter.

2.2 ARM

Originated in the 1980s, the history of ARM started when a company named Acorn Computers looking for a 16-bit microprocessor for their next desktop machine could not find any that suited their needs. Their solution, make their own microprocessor. Based on the Berkeley RISC 1 [9], in 1985 Acorn released the first Acorn RISC Machine (ARM) in 1985, a compact 26-bit RISC core, becoming the first commercial RISC processor in the world. This microprocessor matched or exceeded contemporary CISC processors, like the Intel 80286 processor, while using less than 25,000 transistors, a very small number even for 1985. This established ARM's reputation for efficiency and clean architectural design.

This first ARM architecture used by Acorn, is now know as the ARM version 1 architecture. Since then, the ARM architecture has continued evolving. In 1987, the ARM version 2 introduced coprocessor support. In 1990, the company "Arm Holdings" was founded when the Acorn architecture division spun off as a joint venture with Apple and VLSI Technology. In this document, the term "ARM" (in all caps) has been used and will be used to refers the processor architecture, when referring to the company, it will be by the name "Arm Holdings" [10].

The 3th generation of the ARM architecture included 32-bit addressing, MMU support, and 64-bit multiplier-accumulate instructions. It was implemented in the ARM 6 and ARM 7 cores. The release of these processors marked ARM's entry into the embedded systems market. The 4th generation of the ARM architecture introduced support for the compressed 16-bit Thumb instruction set, which reduced its size.In 1999, the 5th generation of the ARM architecture introduced digital signal processing and Java byte code extensions to the ARM instruction set [11].

The 6th generation of the ARM architecture introduced the SIMD instruction set extension, improved the Thumb instruction set, TrustZone virtualization technology, and multiprocessor support.The 7th generation of the ARM architecture established a consolidated 32-bit instruction set with enhancements including extended SIMD capabilities, improved floating-point support, and the Thumb-2 instruction set for code density optimization. The 8th generation of the ARM architecture marked a generational leap by introducing 64-bit execution while maintaining backward compatibility with 32-bit code of the 7th generation [12].

It is necessary to make a clarification regarding the Arm Holdings company, they were never a microprocessor manufacturing company, as can be understood from the text of this section; they always been a fabless[1] company. Their business model has always been the design of ARM architectures (ARMv4, ARMv5, Cortex families, ARMv8, etc.) and licenses these designs to semiconductor companies, such as STMicroelectronics or NXP, which use them as their products core.

[1]The term "fabless company" refers to a company that designs and markets semiconductors while outsourcing that hardware's fabrication (or fab) to a third-party partner, who produces them in foundries.

Version	Year	Features	Implementations
v1	1985	The first commercial RISC (26-bit)	ARM1
v2	1987	Coprocessor support	ARM2, ARM3
v3	1992	32-bit, MMU, 64-bit MAC	ARM6, ARM7
v4	1996	Thumb	ARM7TDMI, ARM8, ARM9TDMI, StrongARM
v5	1999	DSP and Jazelle extensions	ARM10, XScale
v6	2001	SIMD, Thumb-2, TrustZone, multiprocessing	ARM11, ARM11 MPCore

Table 2.3: History of the ARM architecture
[10] [13]

2.2.1 ARM In Education

Nowadays, ARM is the world's leading microcontroller used in the designing of embedded Systems and can be found in applications ranging from smartphones and IoT devices to automotive and aerospace systems. According to the ARM Architecture Organization, in the fourth quarter of 2020, ARM produced approximately 6.7 billion ARM-based chips, its highest ever [14]. The importance of the ARM architecture in today's market justifies its study in universities, given the high likelihood that students will use it in the job market, but is not the only one.

The simplicity of its design, the RISC-basedinstruction set and its open-source nature make the use of ARM-based processors ideal for teaching embedded systems concepts. Its instruction set allows students to easily understand the relationship between high-level programming constructs, written in C or assembly language, and low-level hardware execution, fostering a deep understanding of processor behavior, register manipulation, and memory management. Additionally, once students master the fundamental ARM instruction set and addressing modes, they can quickly extrapolate this knowledge to various ARM implementations, accelerating the learning curve across the entire ARM ecosystem.

The reasons mentioned above explain why many universities are implementing RISC architecture in their undergraduate and master's degree programs related to embedded systems. For example, the STM32F4-DISCOVERY development board is used as part of the course materials for the Embedded Systems I and II courses in the Master's in Electronic Systems at the UPM.

2.3 Tools

2.3.1 STM32CubeIDE

One of the main objectives of this project is to run firmware designed for real hardware on emulated hardware and make it behave as if it were real hardware. Since an STM32F405 microcontroller will be emulated in this case, a tool capable of generating the appropriate firmware for STM32 microcontrollers is required. The STM32CubeIDE is the official C/C++ development IDE from STMicroelectronics for STM32 microcontrollers and microprocessors [15]. This IDE integrates CubeMX code generation (I^2C/clock/NVIC configuration) and the official ST HAL/LL drivers as a reference standard. Therefore, STM32CubeIDE was selected to develop the firmware for this thesis.

2.3.2 ARM GNU Toolchain

Although the STM32 ecosystem includes STM32CubeMonitor [16], a powerful tool for debugging and real-time monitoring of microcontroller variables, it requires physical hardware and ST-LINK connectivity. While the ultimate goal of this thesis is firmware compatibility with the actual STM32F405 hardware, iterative development demands debugging capabilities during emulation to validate both the firmware logic and the fidelity of the QEMU device model. After investigating the STM32 ecosystem, no solution was found that requires fully virtual environments. Under this context two debugging options were evaluated:

1. **QEMU's built-in GDB server ('-s -S'):** QEMU offers a feature that exposes a GDB stub on TCP port 1234. When used with 'arm-none-eabi-gdb' in the firmware used for emulated execution of the STM32F405 (previously compiled using STM32CubeIDE), breakpoints, steps, register inspection, and memory watchpoints can be used on the emulated STM32F405.
2. **Eclipse CDT GDB plugin:** Eclipse have a plugin for connecting to QEMU's GDB server, that provides a graphical frontend but it requires additional configuration beyond STM32CubeIDE's native ST-LINK focus

Due to its simplicity, zero external dependencies and direct integration with the QEMU workflow, the QEMU integrated GDB server option with command line 'arm-none-eabi-gdb' was selected. The 'arm-none-eabi-gdb' command is part of the ARM GNU Toolchain, the official open-source development suite provided by Arm for bare-metal Cortex-M programming. Apart from including the 'arm-none-eabi-gdb' it also includes: the compiler 'arm-none-eabi-gcc'; the linker 'arm-none-eabi-ld' and the disassembler 'arm-none-eabi-objdump' [17] [18].

2.3.3 System & Main IDE

The entire development environment used for this thesis utilizes Fedora 42 [19] [20] as the operating system. From the start, it was clear that a Linux-based operating system would be used for the thesis development, as Linux provides a native Unix-like toolchain that seamlessly supports STM32 firmware development and QEMU modifications. While Fedora offers certain advantages over other distributions by striking a balance between stability and incorporation of the latest open-source software, there is no specific reason to choose it over any other distribution beyond the developer's personal preference.

Unlike the operating system, the primary IDE used during development belongs to the Microsoft ecosystem. Visual Studio Code [21], a lightweight, cross-platform, and open-source code editor, has been the main IDE used during the project's development due to its ease of use and widespread use in both professional and educational environments.

Chapter 3

Analysis

Although not part of the scope of this Master's thesis, the ultimate goal is the creation of a virtual laboratory as a teaching tool for microcontrollers. This virtual laboratory, which uses QEMU as its emulation platform, aims to emulate development boards with ARM processors and the peripherals used in the UPM laboratories. This Master's thesis aims to contribute to the creation of this teaching tool by incorporating the I^2C protocol into these boards, as well as including one of the peripherals used as an I^2C slave.

This chapter will analyze the selected hardware, the reasons for its selection, and the design decisions made before its implementation in QEMU. It will also analyze the basis used to create the verification firmware—the firmware used to test the behavior of the emulated hardware. While the tools used during the project's development were already mentioned in sectio 2.3 of the previous chapter, due to Its importance, this chapter will also include an analysis of the inner workings of the QEMU emulation tool.

3.1 Hardware

The NUCLEO-F411RE, NUCLEO-F446RE, and STM32F411E-DISCO development boards are among those used by the UPM in certain degree programs, such as the Data Acquisition Systems course in the Data Systems Engineering degree program, or in the Electronic Systems Laboratory of the MUISE. Although different, these development boards share a common element: they all use an STM32F4xxx microcontroller as their main component. The NUCLEO-F411RE [22], NUCLEO-F446RE [23], and STM32F411E-DISCO [24] boards use the STM32F411RET6 [25], the STM32F446RET6 [26], and the STM32F411VET6 microcontrollers, respectively. Different models, but with the same root (stm32f4 in their names), indicate that all three microcontrollers have a 32-bit Arm Cortex-M4 processor. Unfortu-

nately, none of these microcontrollers are directly implemented in the SIVALSE QEMU version. However, the STM32F405 microcontroller is. While this is not surprising, given that they share the same manufacturer and the same Arm Cortex-M4 architecture, an analysis of the reference manuals for all four microcontrollers, specifically regarding the I^2C protocol, reveals that they share a basic I^2C controller architecture. During the analysis, no register-level differences were observed between them, but even if variations existed that went undetected, a QEMU implementation of the I^2C protocol for the STM32F405 could serve as a fundamental basis for future I^2C implementations of these microcontrollers, requiring only model-specific adjustments.

3.1.1 STM32F405

The STM32F405, part of STMicroelectronics' STM32F4 family, is based on a 32-bit ARM Cortex-M4 core with a Floating Point Unit (FPU), support for DSP instructions, an Adaptive Real-Time Accelerator (ART) for 0-wait execution from Flash up to 168 MHz, a Memory Protection Unit (MPU), and 210 DMIPS of performance. This configuration makes it an ideal microcontroller for embedded systems with real-time and intensive processing requirements, such as motor control, power conversion, and multimedia applications. It incorporates up to 1 MB of Flash memory, up to 192+4 KB of SRAM (with 64 KB of CCM), and essential peripherals: three 12-bit ADCs, two DACs, 17 timers, 16-stream DMA, three I^2C ports, multiple USART/SPI/CAN ports, Ethernet MAC, USB OTG HS/FS, and up to 140 fast, 5V-tolerant GPIOs.

STM32F405 I^2C

As explained previously, It has been decided to add emulation of the I^2C STM32F405 communication protocol to QEMU. This microcontroller has three I^2C peripherals (I^2C1, I^2C2, I^2C3), each providing a fully compliant I^2C bus interface. These peripherals automatically manage bus-specific sequencing, protocol management, arbitration, and timing, simplifying firmware development and ensuring robust communication. A Summary of the main capabilities of the STM32F405 [27]:

- I^2C standard compliance, with 7-bit and 10-bit addressing modes.
- Dual-speed modes:
  - Standard mode(Sm), up to 100 kHz.
  - Fast mode(Fm) up to 400 kHz.
- Master-slave operation, with multi-master capability and clock expansion support.

- Hardware packet error checking (PEC) using the CRC-8 polynomial for increased reliability.
- DMA support for efficient data transfers without CPU intervention.
- Dual interrupt vectors: one for successful communication events and one for error conditions.

A design decision made prior to implementation was to focus I^2C development exclusively on master mode, as this represents the operating context in which the STM32F405 communicates with the LIS3DH accelerometer. The emulated microcontroller's firmware initiates all I^2C transactions, while the LIS3DH device model responds as a slave. This design decision simplifies the scope of the emulation while maintaining full functional reliability for the intended use case.

STM32F405 I^2C Registers

The STM32F405 I^2C peripheral operates through a set of memory-mapped registers that control all aspects of bus communication [28]. Understanding this register architecture is fundamental to accurate emulation, as firmware interacts with the peripheral exclusively through register read and write operations. The register set can be functionally categorized into four groups:

- **Control Registers:** The registers I2C_CR1 and I2C_CR2
- **Timing Configuration Registers:** The registers I2C_CCR and I2C_TRISE
- **Data and Address Registers:** The registers I2C_DR and I2C_OAR1
- **Status Registers:** The registers I2C_SR1 and I2C_SR2

Among these register groups, the control registers, status registers, and the I2C_DR register will have the greatest impact on emulation. On the other hand, the I2C_OAR1 register is irrelevant, as it only defines the slave address of the peripheral when the microcontroller is operating in slave mode. Regarding the timing configuration registers, it was decided not to implement their associated logic. This is because QEMU, to emulate the operation of the I^2C protocol, provides a series of functions that execute the different stages of the I^2C communication protocol without considering timing. Therefore, it is not currently necessary to analyze these registers.

For implementation in the emulator, it is crucial to understand the logic of the control and status registers, as well as the I2C_DR register. The I2C_DR register, the data register, functions as a bidirectional data buffer between the firmware and the I^2C shift register. The control registers, I2C_CR1 and I2C_CR2, on the other hand,

serve as the main interface for peripheral control and trigger transitions between the different phases of the I²C protocol. Specifically, the I2C_CR2 register contains the bits that configure clock generation and interrupt control, while the I2C_CR1 register contains, among others, the following bits:

- **PE**, which enables the use of I²C communication itself.
- **START**, which generates the start condition on the bus.
- **STOP**, which is responsible for generating the stop condition on the bus and terminating communication.
- **ACK**, the acknowledgment bit, which controls whether the peripheral generates an acknowledgment pulse after receiving a byte, which is essential for multibyte read operations.

Following the previous explanation, the status registers contain information about certain conditions that have been met during a specific operation and that can be taken into account in subsequent operations. The I2C_SR2 register contains bits with information about the bus status, including the MSL bit, which indicates the current operating mode, the BUSY bit, which reflects bus activity, and the TRA bit, which shows the current data direction. The I2C_SR1 register, on the other hand, provides real-time visibility into communication events and error conditions. The key indicators in this register are:

- **SB:** Set when a START condition (master mode) has been generated.
- **ADDR:** Set when the address byte has been successfully transmitted.
- **BTF:** Set when both the shift register and the data register are empty/full.
- **TxE:** Set when the data register is empty and ready for new data.
- **RxNE:** Set when the data register contains received data.
- **AF:** This is established when no acknowledgment is received from the slave.

STM32F405 I²C Sequences

To ensure maximum fidelity between the I²C protocol of the real hardware and the I²C of the emulated hardware, not only must the individual logic of each register be the same as that of the real hardware, but the behavior of the registers as a whole must comply with the expected generation of events of the I²C protocol.

The STM32F405 microcontroller reference manual defines the transmit and receive sequences necessary to establish I²C communication. These sequences contain

the precise operations and register transitions that the firmware must perform at each stage of the communication. A model emulating the behavior of this I^2C interface must be able to recognize these operations and register transitions and generate the corresponding I^2C events, such as sending the START, STOP, or ACK signals. These sequence are represented in the Annexe C

3.1.2 LIS3DH

Other elements intended for inclusion in the virtual laboratory, and which are part of the UPM laboratory equipment, are all the sensors from the Arduino Sensor Kit [29] by seeedStudio, including the LIS3DH accelerometer. This three-axis accelerometer, designed by STMicroelectronics, which has a digital output with an I^2C interface, was selected in this thesis for integration into QEMU and to test the I^2C interface of the emulated STM32F405 microcontroller.

Characteristics

The LIS3DH [30] is an ultra-low-power, high-performance, three-axis linear accelerometer belonging to the "nano" family, with a standard I^2C/SPI digital serial interface output. The device features ultra-low-power operating modes for advanced energy savings and integrated intelligent functions, as well as a 32-level FIFO that can store up to 32 complete sets of samples.

The LIS3DH provides three-dimensional acceleration measurement with a selectable dynamic range of ±2 g, ±4 g, ±8 g, and ±16 g, depending on the required use case. The data output resolution is 16 bits. The device is capable of generating output data rates (ODR) from 1 Hz to 5.3 kHz, covering applications ranging from low-power real-time monitoring to high-frequency event capture. The current consumption of the LIS3DH varies significantly depending on the operating mode: in full operation mode, it can reach approximately 70 µA, while in low power modes, it drops to as low as 2 µA, a critical feature for battery-powered applications.

Registers

The LSI3DH has 31 eight-bit registers allocated to the address space. These registers are responsible for both device configuration and access to measurement data and status information. Each of these aspects is controlled by a different set of registers. This section of the chapter will analyze the most relevant ones. A table with the accelerometer's memory map can be found in Appendix D.

This STMelectronics accelerometer has three operating modes: low-power mode, normal mode, and high-resolution mode. These operating modes are defined by

setting the LPen bit in CTRL_REG1 and the HR bit in CTRL_REG4, Table 3.1.

Mode	**LPen**	**HR**	**Resolution**	**Characteristics**
Low-Power	1	0	8-bit	Highest power efficiency, 8-bit left-shifted data
Normal	0	0	10-bit	Balanced power/performance, 10-bit left-shifted data
High-Resolution	0	1	12-bit	Maximum precision, 12-bit left-shifted data

Table 3.1: LIS3DH operating modes and resolution

The output data rate is configured using the ODR[3:0] bits of the CTRL_REG1 register, table 3.2. This register also contains the individual enable bits (Xen, Yen, Zen) for each measurement axis.

ODR3	**ODR2**	**ODR1**	**ODR0**	**Power Mode / Output Data Rate**
0	0	0	0	Power-down
0	0	0	1	HR/Normal/Low-power: 1 Hz
0	0	1	0	HR/Normal/Low-power: 10 Hz
0	0	1	1	HR/Normal/Low-power: 25 Hz
0	1	0	0	HR/Normal/Low-power: 50 Hz
0	1	0	1	HR/Normal/Low-power: 100 Hz
0	1	1	0	HR/Normal/Low-power: 200 Hz
0	1	1	1	HR/Normal/Low-power: 400 Hz
1	0	0	0	Low-power: 1.60 kHz
1	0	0	1	HR/Normal: 1.344 kHz; Low-power: 5.376 kHz

Table 3.2: lis3dh data rate configuration

As explained in its specifications, the LIS3DH supports four measurement ranges: ±2 g, ±4 g, ±8 g, and ±16 g. These are configured using bits FS[1:0] in the CTRL_REG4 register. The range affects both the maximum measurable acceleration and the sensitivity. Table 3.3 summarizes this:

FS[0:1]	**Range**	**Range (mg)**	S_0 **Normal**	S_0 **High-Res**	S_0 **Low-Power**
00	±2g	±2000 mg	4 mg/LSB	1 mg/LSB	16 mg/LSB
01	±4g	±4000 mg	8 mg/LSB	2 mg/LSB	32 mg/LSB
10	±8g	±8000 mg	16 mg/LSB	4 mg/LSB	64 mg/LSB
11	±16g	±16000 mg	48 mg/LSB	12 mg/LSB	192 mg/LSB

Table 3.3: lis3dh full-scale ranges and sensitivities

The LSI3DH stores acceleration measurement data in the output data registers: OUT_X_L/H, OUT_Y_L/H, and OUT_Z_L/H. A very important feature of these reg-

isters is that their values are expressed in left-justified two's complement. Regarding the interruptions, the LIS3DH has two independent interrupt lines, INT1 and INT2. These can be configured to signal different types of events, significantly reducing the computational load on the host processor. The device supports three categories of interrupts: data interrupts (data ready and FIFO status), inertial event interrupts (axis movement, wake-up, freefall, and 6D/4D orientation detection), and click detection interrupts. The CTRL_REG3 and CTRL_REG6 control registers determine which events are routed to INT1 and INT2, respectively.

3.2 STM32 HAL

STMicroelectronics provides a software framework called the STM32 Hardware Abstraction Layer (HAL) [31]. This framework enables programmers to deal with STM32 peripherals by providing methods that abstract the peripheral's complexity while maximizing code portability throughout the full STM32 product line. Firmware developers can setup and connect with the peripherals using HAL methods like 'HAL_I2C_Mem_Read()' and 'HAL_I2C_Mem_Write()' instead of directly modifying hardware registers. This allows the same application code to run on several STM32 variations with little change.

In the context of this thesis, the LIS3DH emulation model is designed to achieve full API compatibility with the physical sensor's HAL drivers. By faithfully implementing the I^2C sub-addressing protocol, register semantics, and interrupt behavior, as specified in the STMicroelectronics datasheet, the emulated sensor allows unmodified STM32CubeF4 HAL code to run identically in the QEMU environment as it would in real hardware.

3.3 QEMU

QEMU is an open-source, is a generic and open source machine emulator and virtualizer designed to emulate entire hardware architectures [32]. Since its creation by Fabrice Bellard in 2005 [33], many features and improvements have been added to QEMU, transforming it into a great virtualization solution that supports a wide range of CPU architectures—including x86, ARM, RISC-V, PowerPC, and MIPS—making it a common choice in embedded systems development, validation, and educational environments. Nowadays, QEMU offers multiple modes of operation, each serving distinct use cases:

- **Full-system emulation:** In this mode QEMU provides complete virtual machine emulation with virtual CPU, memory, and emulated peripherals.

- **User-mode emulation:** Running individual applications compiled for different architectures than the guest system. In this mode QEMU emulates only the CPUs. It executes guest instructions, captures system calls, and sends them to the host kernel.
- **Hardware-virtualization:** QEMU use hardware extensions like KVM or Xen to emulate the guest code, with near-native performance, though requiring the guest and host to share the same instruction set architecture.

Regardless of the mode, at the highest level QEMU operates as a conventional userspace[1]1 application on the host operating system, Figure 3.1. In the current thesis, due the necessity to emulates a entire developer board containing the STM32F405 microcontroller, including the memory, the peripherals, the buses and the firmware, QEMU is executed in full system emulation mode.

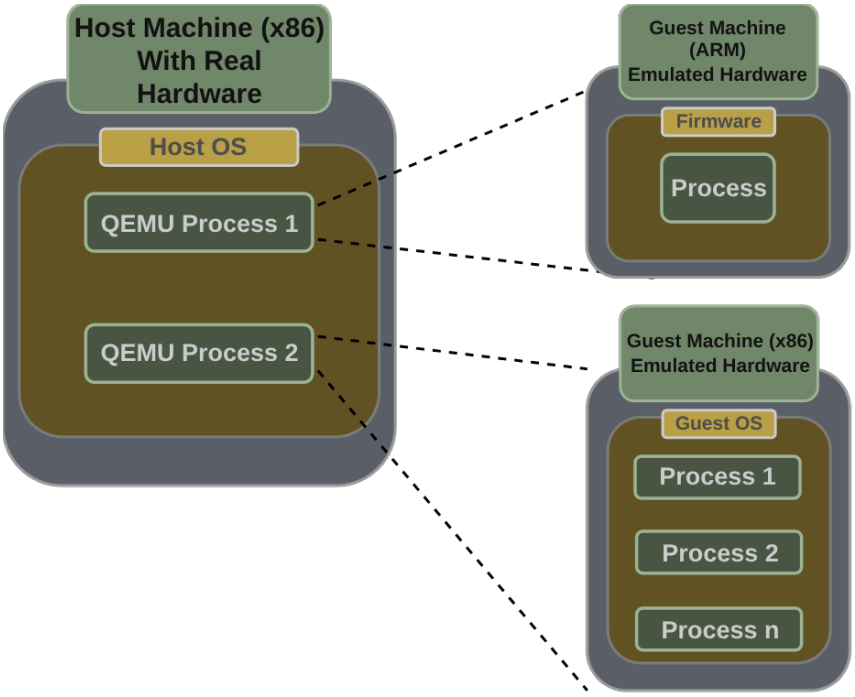


Figure 3.1: Full system emulation architecture.

The device emulation is the mechanism that represents hardware as software objects executing on the host CPU. Unlike functional simulation—which attempts to model the behavior of the device at a higher level of abstraction—QEMU's device emulation target for cycle-accurate and register-accurate fidelity. Understanding the inner workings of QEMU and how to create and integrate new devices and buses is essential for developers emulating custom hardware. Since this thesis implements both the STM32F405 microcontroller's I^2C controller and the LIS3DH accelerometer as an I^2C slave, understanding QEMU's internal structure and its most important elements and subsystems is It is considered necessary.

[1]The term user space (or userland) refers to all code that runs outside the operating system's kernel

3.3.1 Machine Code Translation

As mentioned, QEMU at its core employs binary translation technology, allowing it to interpret and execute machine code compiled for one type of CPU on a completely different CPU. A typical binary translation converts the guest CPU's (also known as virtual CPU or vCPU) entire binary file into the host CPU's binary file using some method, and then executes the converted binary on the host CPU. However, this approach presents a significant problem, any modifications made to the guest program require, once again, a complete binary conversion. This wouldn't be a problem for small programs, but it is for larger ones. Longer conversion times may be acceptable for large guest programs if the conversion only needs to be done once. But, when frequent modifications to the guest program is need, like in embedded software development, this approach becomes impractical.

QEMU solves this problem by not implementing a traditional translation technology, instead use dynamic binary translation (DBT). This type of translation does not convert the entire vCPU program at once. it only converts the necessary parts of the program and executes them. For example, when booting a guest OS, only the parts required for booting are converted and executed. To be more specific, QEMU advances the program counter for the vCPU step by step, converting one vCPU instruction into one or more host CPU instructions and executing them, then advancing the program counter again, converting, executing, and repeating this process. With this method, there is no need to convert the parts that are not executed.

The inefficiency of translating the entire program binary from the vCPU is evident. However, translating and executing only one guest instruction at a time incurred a considerable time overhead. This is due to context management. QEMU is a regular program running on the host CPU, so to execute the instructions translated from the vCPU, it first needs to save the state of the host CPU's registers and memory context. Furthermore, once the translated instruction finishes executing, the saved state of the host CPU must be restored. Because of this inefficiency, DBT parses a short chunks of code, translates it, and caches the resulting sequence. This way, code is only translated as it is discovered and when possible, and branch instructions are configured to point to code that has already been translated and saved.

QEMU's DBT takes the named of the Tiny Code Generator (TCG), and the basic code fragments it analyzes are called Translation Blocks (TB). In the case of QEMU's TCG, the TBs are defined by using any instruction that changes the control flow of the program—jump or return instructions, interruptions or peripherals accesses—as boundaries [34] [35]. The translation performed by the TCG consists of two phases: converting the host code into a sequence of micro-operations and turning these micro-operations into executable code [36]. The figure 3.2 illustrates the general architecture and workflow of the TCG.

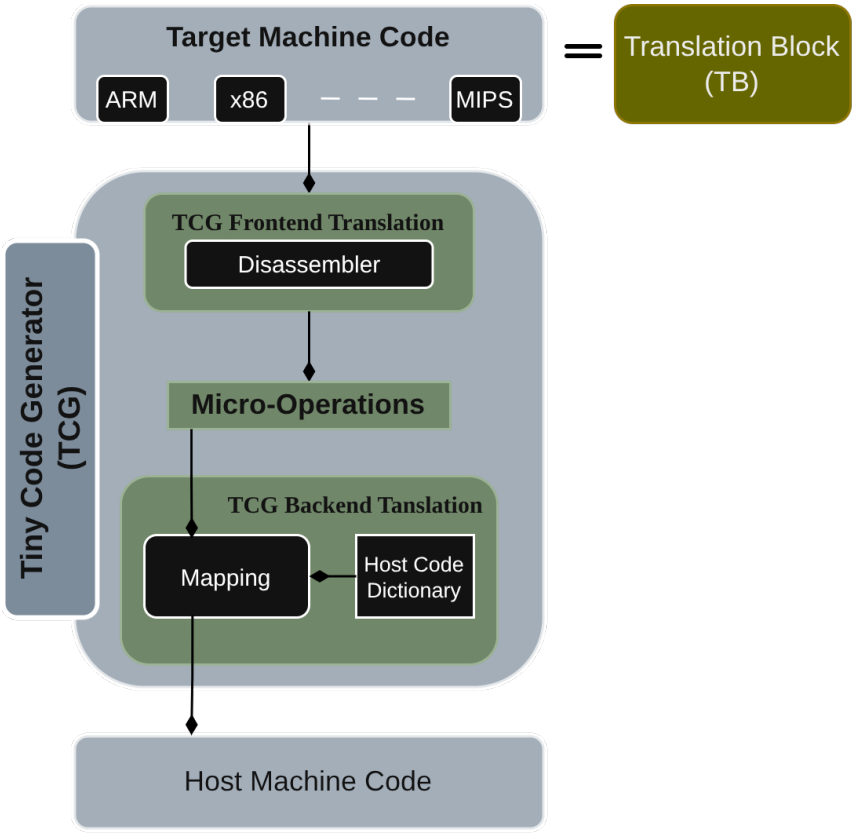


Figure 3.2: Tiny Code Generator

In the first phase, the TCG's frontend section decomposed each guest instruction from the corresponding TB into equivalent sequences of QEMU-specific intermediate representations (IRs), called micro-operations. Conceptually, these micro-operations are like an assembly language designed specifically for QEMU's compilation infrastructure.

In the second phase, the TCG's back-end section converts this intermediate representation into native host machine code, which is then directly executable on the host processor [37] [38]. The TCG maintains a dictionary of micro-operations. This dictionary contains a mapping between micro-operations and the equivalent sequences of host architecture binary code. This binary code performs the task of the corresponding micro-operation when run on the host processor. The TCG cycles through each micro-operation contained in the translation block and emits the corresponding sequence of host machine instructions by using the dictionary. The result is a block of host machine code that performs the same operations as the guest machine code [36].

The QEMU developers' decision to separate front-end and back-end translation was a wise one, as it offers significant advantages for QEMU's scalability. Thanks to this decision, to support a new host CPU architecture, developers only need to

implement a new front-end translator, while the back-end translation logic remains unchanged. Conversely, supporting QEMU on a new host architecture only requires back-end development, without affecting compatibility with that architecture. This modularity is the primary reason for QEMU's scalability across various hardware platforms and CPU architectures.

3.3.2 QEMU's Internals

Internally, in its early versions, QEMU faced a very complex challenge: designing a modular architecture that could manage diverse devices, each with different register layouts, timing behaviors, and interrupt patterns. To address this challenge QEMU was presented as a model of multiple abstraction layers, each one in charge of a critical part of the process and hiding lower-level details from the others (Memory/IO Abstraction Layer, Device Modeling Abstraction Layer, Device Lifecycle Abstraction Layer). To implement these layers of abstraction, and with the goal to achieve a modular and a maintainable code, QEMU's internal architecture was structured as a set of different subsystems, working together to manage device emulation, memory modeling, interrupt handling, and system orchestration [39].

Each major subsystem implements one or more APIs that define how other components interact with it. This approach allows developers to implement new hardware within the QEMU codebase without having to understand the internal details of each subsystem, thus achieving modularity and code reusability. Understanding these APIs is essential for contributing to QEMU or developing custom emulators, particularly when implementing new devices and buses [40].

This section of the chapter explores the subsytems ant their APIs that, in the developer's opinion, are most important to understadn when implementing new hardware in QEMU code: the QEMU Object Model (QOM), the QEMU Device Model (qdev), and the QEMU memory APIs. This exploration aims to provide sufficient context on the operation of each so that, by the time te reader reach the implementation in Chapter 5, the concepts and procedures used there will be familiar.

QEMU Object Model (QOM)

The QEMU Object Model (QOM) subsystem is QEMU's object-oriented framework. This subsystem provides a device modeling abstraction layer by implements a type system using pure C which, despite C's lack of native object-oriented programming (OOP) features, allows dynamic type registration, inheritance, and runtime polymorphism, providing [41] [42]:

- **Object-oriented programming:** Without requiring an OOP language like

C++.

- **Dynamic type registration:** Types can be registered at runtime without recompilation.
- **Inheritance and polymorphism:** Types inherit from parent types (single inheritance) and override methods.
- **Runtime reflection:** All objects and properties are discoverable via standardized queries.
- **Dynamic properties:** Each object instance has configurable, typed properties.
- **Composition tree:** Hierarchical object relationships enabling complex system modeling.

Before the introduction of QOM, QEMU development was fragmented. Each board implemented its devices independently, requiring duplicate code; device configurations were hardcoded; and extending QEMU with new hardware was cumbersome. The introduction of the QOM subsystem resolved these problems by unifying device management through the implementation of:

- **Uniform object model:** All devices are Objects with properties.
- **Runtime type registration:** Types defined via TypeInfo at startup.
- **Common instantiation interface:** Devices created uniformly by type name.
- **Dynamic configuration:** Properties accessible and modifiable via a standardized API.

QOM's type system is foundational to QEMU device modeling. Every emulated peripheral (timers, ADCs, I^2C controllers) is a QOM Object instance with an ObjectClass defining its behavior. This abstraction decouples device logic from QEMU core, enabling researchers to develop and validate new device models—such as the STM32F405 I^2C controller developed in this thesis—within a consistent, well-defined framework. The figure 3.3 illustrates the scope of QOM's management within QEMU.

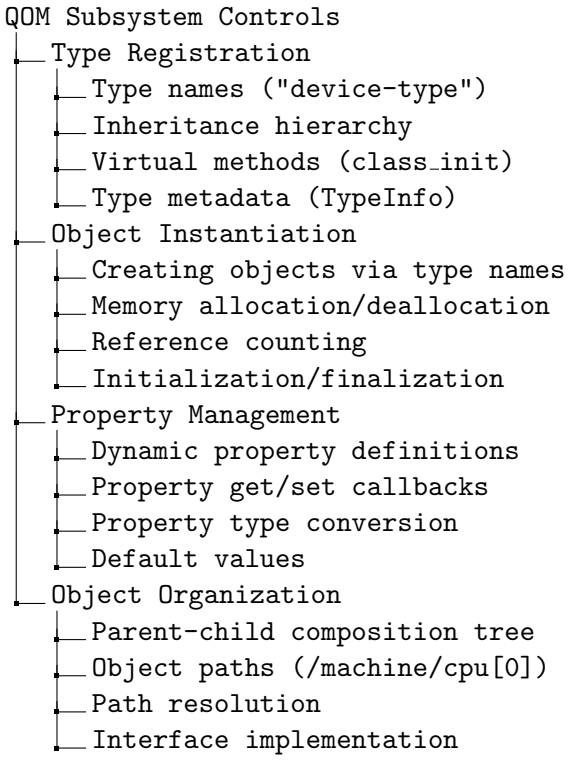


Figure 3.3: QOM Management Scope

There are several important elements of the QOM API that were used during the implementation and development of this thesis and that need to be understood. Among these elements, the ones most frequently mentioned throughout the document are the structures `ObjectClass`, `Object`, `ObjectProperty`, and `TypeInfo`. Since their explanation can be somewhat technical, it has been included in an appendix at the end of the document. Appendix E.

QEMU Device Model (Qdev)

While the QOM provides the infrastructure for the creation and the handling of objects in QEMU, these are generic and they do not specifically address the requirements of hardware device modeling. As result, QEMU Device API (QDev) was designed to covered this gap. This QEMU's API is a specialized framework built on top of QOM, because of this, each QDev device is itself a QOM object, meaning it inherits QOM's type system, introspection, and dynamic property mechanisms.

Qdev functions as an abstraction layer for the device lifecycle, unifying the description, creation, connection, and configuration of the emulated hardware components, and ensuring that every device, regardless of its specific peripheral function, follows consistent patterns for initialization, property management, lifecycle stages,

and system integration. The device-specific capabilities that Qdev introduce for hardware emulation includes: DeviceClass, Lifecycle Stages, Bus Abstraction, GPIO/IRQ System, Reset Propagation, Hotplug Support.

Most of the capabilities mentioned are achieved through several layers of abstraction in order to standardize and simplify the creation of emulated hardware to the developer. Due to the associated complexity, it is important for developers to understand qdev's layers of abstraction before reading or writing code; not doing it, could lead to a spiral of confusion about how and why things are done in QEMU. The device lifecycle is the first thing Qdev applies, providing a guide with several standardized phases to ensure that the firmware running on QEMU can reliably interact with devices through predictable boot sequences and initialization patterns. The Figure 3.4 show this process.

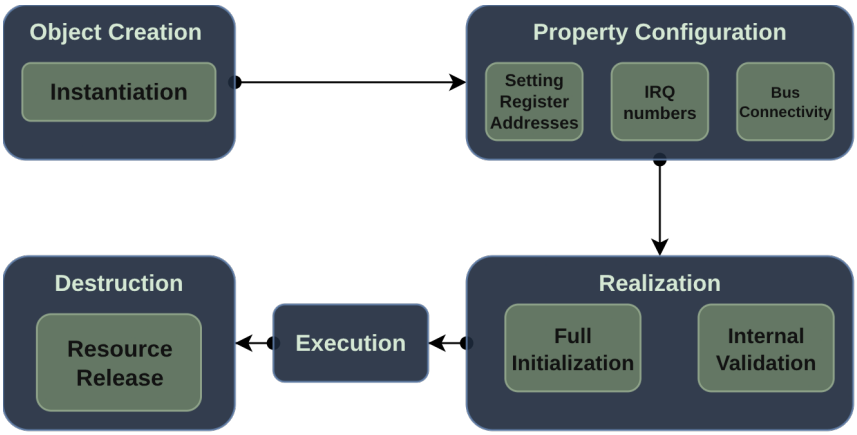


Figure 3.4: Devices Lifecycle

The second element provided by QDev is a bus abstraction layer, which will be especially relevant later in this thesis for emulating the STM32F405 address map. In QEMU, thanks to this bus abstraction layer, devices do not exist in isolation but are organized hierarchically and connected to buses. These buses manage address allocation, establish parent-child relationships, and handle address translation for memory-mapped I/O. This is fundamental for emulating microcontrollers with multiple peripherals, each occupying different memory regions and responding to specific addresses on the bus. Table 3.4 shows a list of common QEMU buses and their purpose.

Bus Type	**Purpose**
System Bus (sysbus)	Default; memory-mapped devices
I^{2}C Bus	I^{2}C protocol; slave devices
PCI Bus	Peripheral Component Interconnect
SPI Bus	Serial Peripheral Interface

Table 3.4: QEMU Bus Types and Their Purposes

Fourth, QDev implements reset propagation, allowing a system reset command to cascade through the entire device tree, ensuring all peripherals return to their initial states in the correct order—essential for firmware that relies on peripheral reset behavior. And finally, QDev also supports hotplug, permitting devices to be dynamically added or removed at runtime, though this feature is less critical for bare-metal microcontroller emulation than for system-level scenarios. Similar to the QOM API, in Annex E there are several key elements of the QDev API that will be used and mentioned throughout the thesis.

QEMU Memory

At its highest level, QEMU is simply another user-space application on the host operating system. However, unlike other user-space applications, QEMU must manage the modeling of a guest machine's entire memory hierarchy, including RAM, memory-mapped I/O (MMIO) devices, ROM, and other specialized controllers. To accomplish this monumental task, QEMU has a memory subsystem, which in turn has an API. This memory API provides several abstraction layer that QEMU uses to model and manage the memory and I/O address spaces of virtual machines. Its main responsibilities are:

1. **Address Space Representation:** Organize memory into a hierarchy that reflects the guest's physical address space
2. **Memory Type Support:** Model different memory types (RAM, ROM, MMIO, IOMMU, containers, aliases).
3. **Performance:** Maintaining translation caches and utilizing efficient lookup structures to minimize address translation overhead.
4. **Address Translation:** Efficiently translate guest addresses to host memory pointers or device callbacks.
5. **Dynamic Topology:** Support dynamic changes to the memory map (region add/remove, hotplug).
6. **Acceleration Support:** Provide hooks for KVM, Xen, and other accelerators.
7. **Migration Support:** Track memory regions for state migration.

There are two important concepts in the QEMU memory model that must be understood: **MemoryRegions** and **AddressSpaces**. Understanding them is essential for a better grasp of the memory hierarchy and how accesses to guest memory are transformed into host operations. MemoryRegions are the fundamental building blocks that represent individual memory fragments (RAM, MMIO regions, ROM,

aliases, containers) that form a hierarchical tree structure, while an AddressSpace is a complete graphical window that provides a CPU or device with access to a specific root MemoryRegion and its entire subtree. MemoryRegions define what memory exists; AddressSpaces define who can access it and how. There are multiple types of memory regions, all represented by a single C type `MemoryRegion` [43]:

- **RAM Regions:** Represent ordinary guest RAM.
- **MMIO Regions:** Represent memory-mapped I/O devices accessed via callback functions. Each read or write causes a callback to be called on the host.
- **ROM Regions:** Represent read-only memory (firmware, bootloaders). Reading from ROM accesses host memory directly, but writes either are silently ignored or invoke callbacks (in ROM device variant).
- **IOMMU Region:** An IOMMU region translates addresses of accesses made to it and forwards them to some other target memory region. As the name suggests, these are only needed for modelling an IOMMU, not for simple devices.
- **Container Regions:** Represent hierarchical groupings of other memory regions, organizing complex address spaces. Useful for grouping several regions into one unit. For example, a PCI BAR may be composed of a RAM region and an MMIO region.
- **Alias Region:** Allow a subsection of one MemoryRegion to be mapped at a different location in the address space, enabling memory remapping without duplication. Aliases reference an offset and size within their target region.
- **Reservation region:** Represent a reservation region is primarily for debugging. It claims I/O space that is not supposed to be handled by QEMU itself. The typical use is to track parts of the address space which will be handled by the host kernel when KVM is enabled.

Chapter 4

Design

Following the analysis developed in the previous chapter (Chapter 3), and after a series of design decisions, the final architecture of this master's thesis is shown in Figure 4.1. This chapter aims to explain the design decisions made, their justification, and how they have affected the different modules of the project. All these design decisions documented here will lay the foundation for the implementation described in Chapter 5.

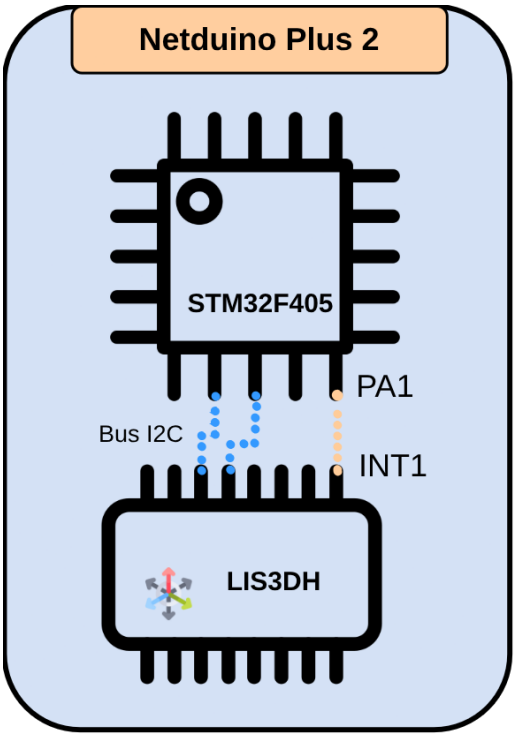


Figure 4.1: General architecture of the project

4.1 Design Considerations

- **I²C Master-Only Mode:**

One of the initial design decisions was to focus the development of the STM32F405's I²C interface exclusively on master mode. This is because, in the educational context in which it will be used, there is no need to use a microcontroller as an I²C slave. Furthermore, the LIS3DH sensor does not have a master mode, so this decision defines the operational context in which the STM32F405 communicates with the LIS3DH accelerometer. This decision simplifies the scope of the emulation while maintaining full functional reliability for the intended use case.

- **Timing Register Abstraction:**

QEMU implements the I²C protocol through functions that abstract the different events of the I²C protocol (Code 4.1), without requiring cycle-level clock generation. This abstraction significantly reduces implementation complexity and preserves functional correctness for firmware testing. To emulate the STM32F405 I²C protocol, this means that it is not necessary to implement the timing logic controlled by the I2C_CCR and I2C_TRISE registers.

code 4.1: `qemu i2c functions`

```
int  i2c_bus_busy(I2CBus *bus);
int  i2c_start_transfer(I2CBus *bus, uint8_t address, bool is_recv)
   ;
int  i2c_start_recv(I2CBus *bus, uint8_t address);
int  i2c_start_send(I2CBus *bus, uint8_t address);
int  i2c_start_send_async(I2CBus *bus, uint8_t address);
void i2c_schedule_pending_master(I2CBus *bus);
void i2c_end_transfer(I2CBus *bus);
void i2c_nack(I2CBus *bus);
void i2c_ack(I2CBus *bus);
void i2c_bus_master(I2CBus *bus, QEMUBH *bh);
void i2c_bus_release(I2CBus *bus);
int  i2c_send(I2CBus *bus, uint8_t data);
int  i2c_send_async(I2CBus *bus, uint8_t data);
uint8_t i2c_recv(I2CBus *bus);
```

- **Netduino Plus 2 as a Development Board for the STM32F405 Microcontroller:**

Unlike other emulators, QEMU fully emulates a hardware platform; it doesn't offer the option to emulate only MCU models independently. This limitation created the need to find or implement a development board model in QEMU that used the STM32F405 microcontroller. Fortunately, QEMU has a pre-implemented minimalist model of the Netduino Plus 2 electronic development board, based on the STM32F405RG microcontroller, which has saved users from having to develop

a new board to implement the microcontroller.

- **DRDY-Only Interrupt Implementation:**

As explained during the analysis of LIS3DH in section 3.1.2, this accelerometer has several interrupts. Of all the interrupts it offers, only the data-ready (DRDY) interrupt logic could be implemented. While the goal was to implement the logic for all interrupts, the development began with the DRDY interrupt logic due to its simplicity and because it represents the most common interrupt mode in embedded applications. Unfortunately, due to time constraints, the logic for the remaining interrupts could not be implemented. These omissions do not affect the basic functionality required for typical STM32 firmware development scenarios, where DRDY polling or simple data ready interrupts are sufficient.

- **QOM as interface:**

Conceived as a temporary solution until its full implementation in MICROLAB, the LIS3DH accelerometer model has been designed to allow runtime access to acceleration values for each axis. This has been achieved by defining QOM properties in the model. As explained during the QEMU analysis, the QOM API allows the creation of dynamic properties that can be modified at runtime. Thanks to these properties, the developer can inject known acceleration values using commands, such as those described in the Figure 4.2, to test the accelerometer's functionality with known values.

```
(qemu) qom-set /machine/soc/i2c[0]/i2c1/child[0] accel-x 500
(qemu) qom-set /machine/soc/i2c[0]/i2c1/child[0] accel-y -300
(qemu) qom-set /machine/soc/i2c[0]/i2c1/child[0] accel-z 980

(qemu) qom-get /machine/soc/i2c[0]/i2c1/child[0] accel-x
500
(qemu) qom-get /machine/soc/i2c[0]/i2c1/child[0] accel-y
-300
(qemu) qom-get /machine/soc/i2c[0]/i2c1/child[0] accel-z
980
```

Figure 4.2: access to qom properties

4.2 I²C Controller Design

As explained during the analysis, the I²C controller model of the STM32F405 microcontroller must be able to recognize the specific sequences of register read and write operations (Figures C.1 & Figure C.2) that the firmware executes on the real hardware to generate the I²C protocol events. This condition must be met for the emulated hardware to be able to execute the same I²C events in QEMU.

The analysis shows that the generation of I²C events in the controller model depends not only on the current state of the registers but also on the previous I²C events. This implies that, in addition to implementing the individual logic for each register, a finite state machine (FSM) must be implemented, triggered by the register reads and writes.

After an extensive reading of the I²C protocol in the STM32F405 microcontroller reference manual, and following the sequences of Figures C.1 and Figure C.2 which are also shown in the same manual, the state machine of Figure 4.3 was reached. In the Table 4.1 are explained every state.

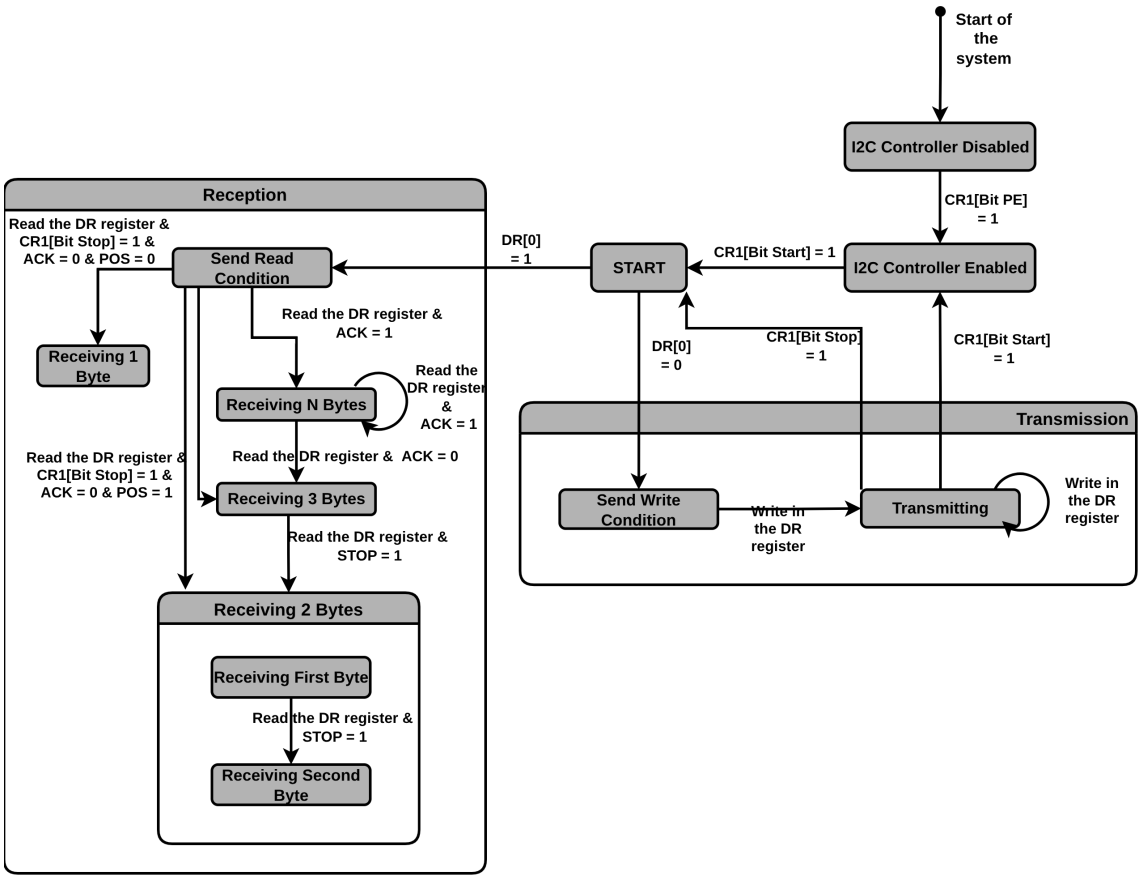


Figure 4.3: stm32f405 i2c controller fsm

Current State	Transition Condition	Next State	Output Action
DISABLED	CR1[PE] = 1 (Peripheral Enable)	IDLE	Initialize FSM
IDLE	CR1[START] = 1	START	Set MSL/BUSY, set SB flag
START	DR write (R/W=0)	S_WRITE	i2c_start_transfer(), set ADDR/TXE/BTF
S_WRITE	DR write	TRANSMITTING	i2c_send(), set TXE/BTF
TRANSMITTING	DR write	TRANSMITTING	i2c_send(), set TXE/BTF
TRANSMITTING	CR1[START] = 1	START	Repeated START condition
TRANSMITTING	CR1[STOP] = 1	IDLE	i2c_end_transfer(), clear flags
START	DR write (R/W=1)	S_READ	i2c_start_transfer(), set ADDR/RXNE/BTF
S_READ	DR read + CR1[STOP]=1 + ACK=0 + POS=0	RECV_1BYTE	i2c_recv(), set RXNE/BTF
S_READ	DR read + CR1[STOP]=1 + ACK=0 + POS=1	RECV_2BYTES_FIRST	i2c_recv(), set RXNE/BTF
S_READ	DR read + ACK=0 + POS=0	RECV_3BYTES	i2c_recv(), set RXNE/BTF
S_READ	DR read + ACK=1	RECV_NBYTES	i2c_recv(), set RXNE/BTF
RECV_2BYTES_FIRST	DR read + STOP=1	RECV_2BYTES_SECOND	i2c_recv() (last byte), generate NACK
RECV_3BYTES	DR read + STOP=1	RECV_2BYTES_FIRST	Transition to 2-byte receive mode
RECV_NBYTES	DR read + ACK=1	RECV_NBYTES	i2c_recv(), continue NACK sequence
RECV_NBYTES	DR read + ACK=0	RECV_3BYTES	Prepare for 3-byte special case
RECV_NBYTES	DR read + STOP=1 + ACK=0	RECV_2BYTES_FIRST	Final byte handling
RECV_1BYTE	STOP=1	IDLE	i2c_end_transfer()
RECV_2BYTES_SECOND	STOP=1	IDLE	i2c_end_transfer()

Table 4.1: STM32F4 I2C Master Mode Finite State Machine Transitions

4.3 LIS3DH Design

The LIS3DH accelerometer model to be emulated in QEMU has been designed as an I^2C slave device, faithfully reproducing the register map and, to a large extent, the configuration logic for all operating parameters (data rate, sensitivity, etc.) defined in the STMicroelectronics datasheet, which were already analyzed in subsection 3.1.2 of the analysis chapter (Chapter 3 of this thesis). All of this aims to ensure that the firmware running on the emulated STM32F405 can configure and interact with the LIS3DH identically to the physical hardware.

The logic of the configuration registers has been a crucial element. Although different options were considered, a mechanism was ultimately designed whereby any modification to the control registers immediately recalculates the operating parameters. The following specific functions have been developed to perform the calculation of the internal parameters:

code 4.2: lis3dh parameters functions

```
static void lis3dh_set_So(LIS3DHState *src);
static void lis3dh_set_acc_range(LIS3DHState *src);
static void lis3dh_set_acc_shifts(LIS3DHState *src);
static void lis3dh_set_operating_mode(LIS3DHState *src);
static void lis3dh_set_odr(LIS3DHState *src);
static void lis3dh_set_fscale(LIS3DHState *src);
```

One design decision made was to allow developers to directly inject known acceleration values into each axis using QOM properties. These properties are accel-x, accel-y, and accel-z, and represent the acceleration of each axis in milligravity units. The values these properties receive from the developer cannot be directly assigned to the output registers (OUT_X_L/H, OUT_Y_L/H, and OUT_Z_L/H); they must first undergo a three-stage conversion process:

- **Range Validation :** Input clamped to current FS limits (e.g., ±2000 mg for ±2g mode).
- **Sensitivity Scaling:** Milligravity value divided by mode/FS-specific sensitivity coefficient.
- **Bit-Justification :** Resulting 16-bit value left-shifted by mode-specific offset (4 bits for High-Res, 6 for Normal, 8 for Low-Power) to match datasheet register layout.

This conversion process ensures that the output registers contain bit-precise representations that match the behavior of the physical sensor, allowing the HAL STM32 controllers to correctly avoid modifying the acceleration data.

Chapter 5

Implementation

This chapter presents the practical realization of the design decisions documented in Chapter 4. The implementation encompasses three primary components integrated into the QEMU emulation framework: the STM32F405 System-on-Chip (SoC) model extension, the STM32F4xx I^2C controller device, and the LIS3DH accelerometer I^2C slave device. Each component has been developed following QEMU's established architectural patterns, leveraging the QOM and qdev frameworks analyzed in the sections 3.3.2 and 3.3.2.

Although this chapter repeatedly mentions several code fragments, for readability reasons, several have not been directly included. However, if needed, the final code for this thesis can be found in the following GitHub repository:

Repository With All The Code Of This Thesis

5.1 Development Environment

Before starting with the implementation and development of the different devices, it is important to discuss the initial step, which is the installation of QEMU and its initial configuration.

5.1.1 QEMU Version & installation

The core of this thesis project lies in the open-source tool QEMU. This thesis uses a customized version of QEMU developed by the SIVALSE project, based on the official QEMU version 9.1.50, capable of emulating the STM32F4 architecture (at the time of writing, the official QEMU version is 10.2.50). This customized version of QEMU is the same one used by MICROLAB to create virtual labs. The

work carried out in this thesis extends the basic structure of the STM32F405 SoC in this customized version of QEMU by integrating the I2C controller.

```
$ mkdir qemu-new
$ git clone https://github.com/DIE-SILVASE/qemu_new.git qemu-new
```

Figure 5.1: qemu installation

5.1.2 Build System Configuration

QEMU utilizes the Meson build system in conjunction with Ninja for compilation. To integrate the new device models, modifications were required to multiple `meson.build` files across the QEMU source tree, Figure 5.2:

- **hw/arm/meson.build**: Added `stm32f405_soc.c` to the ARM-specific hardware compilation targets.
- **hw/i2c/meson.build**: Added `stm32f4xx_i2c.c` to the I2C subsystem compilation targets.
- **hw/sensor/meson.build**: Created new sensor subsystem directory and added `lis3dh.c` to compilation targets.

```
qemu/
└── hw/
    ├── arm/
    │   ├── stm32f405_soc.c (SoC container)
    │   ├── stm32f405_soc.h
    │   ├── netduinoplus2.c (Machine definition)
    │   └── meson.build
    ├── i2c/
    │   ├── stm32f4xx_i2c.c (I2C controller)
    │   ├── stm32f4xx_i2c.h
    │   ├── stm32f4xx_types_i2c.h
    │   └── meson.build
    └── sensor/
        ├── lis3dh.c (Accelerometer)
        ├── lis3dh.h
        ├── lis3dh_types.h
        └── meson.build
```

Figure 5.2: qemu project architecture

This organization ensures that each device type is located in its logically appropriate subsystem, facilitating code discovery and maintenance.

5.1.3 Compilation and Testing Workflow

This thesis emulates a Netduino Plus 2 development board to test the new features of the STM32F405RG microcontroller. This board uses a 32-bit ARM architecture, so before compiling, QEMU is configured with the `arm-softmmu` flag, which ensures that only 32-bit ARM architectures are compiled and unnecessary ones are omitted.

```
$ mkdir -p bin/debug/native

$ cd bin/debug/native

$ ../../../configure --target-list=arm-softmmu \\
     --enable-debug  \\
     --disable-strip \\
     --extra-cflags='-g'

$ make -j$(nproc)

$ cd ../../..
```

Figure 5.3: qemu build

The flags `--enable-debug` and `--disable-strip` are used to enable GDB-based debugging of the STM32F405 SoC emulation. This allowed inspection of the device state (interrupt registers, I2C transaction registers) and hardware-firmware interaction at runtime.

Once installed and configured for the requirements of this project, QEMU offers a wide range of options for hardware emulation. For example, the command shown in Figure 5.4 includes several debug flags that have been essential throughout development, enabling debugging of both QEMU and the firmware.

```
sudo ./qemu-system-arm \
-M netduinoplus2 \
-kernel firmware_test_1.elf \
-d trace:i2c_event \
-D debug.log \
-serial stdio \
-s -S
```

Figure 5.4: QEMU command for debugging

The `-s` and `-S` flags are typically used together during debugging:

- `-s`: Starts a **GDB server** in QEMU on TCP port 1234. This option allows the use of the `arm-none-eabi-gdb` tool to debug the firmware as follows:

code 5.1: Connecting GDB to the QEMU GDB server

```
arm-none-eabi-gdb firmware_test_1.elf
(gdb) target remote :1234
```

- `-S`: Prevents the CPU from starting automatically. QEMU loads the firmware and then halts at reset. Execution only begins when GDB is instructed to `continue` or `step`. This behavior is particularly useful for inspecting registers, setting breakpoints at `main()`, and examining the initial system state before any code runs.

The `-serial stdio` option redirects the emulated UART (typically USART1 or USART2, depending on the machine) to the host standard input and output. In practice, all `printf()` calls from the STM32 firmware routed through UART appear in the terminal where the QEMU command is executed.

5.2 STM32F4xx I²C Controller Implementation

The STM32F4xx I²C controller emulation represents the most complex component of this implementation due to the intricate state machine required to translate firmware register operations into I2C bus events, Figure 4.3. This section details the device structure, register implementation, finite state machine design, and integration with QEMU's I2C bus framework.

5.2.1 Data Structures & Type Definitions

The I2C controller model comprises three files: `stm32f4xx_i2c.c`, `stm32f4xx_i2c.h`, and `stm32f4xx_types_i2c.h`. The implementation file (`.c`) contains all functions that define the controller's behavior and enable QEMU to instantiate it. The header files (`.h`) define the C data structures that compose the controller model, encapsulating both the visible hardware state (registers accessible via MMIO) and the internal logical state required to emulate the I2C protocol sequence.

Core Data Structure: `STM32F4XXI2CState`

The principal structure is `STM32F4XXI2CState`, declared using QOM API macros to represent a complete I2C controller instance. The Code 5.3 shows its declaration.

code 5.2: `STM32F4XXI2CState` structure definition

```
/* STM32 I2C State struct requirements */
typedef struct STM32F4XXI2CState
{
    /* <private> */
    SysBusDevice parent_obj;

    /* <public> */
    MemoryRegion iomem;
    I2CBus *bus;
    qemu_irq irq_event;
    qemu_irq irq_error;

    char *bus_name;

    /* for internall logic */
    stm32f4xx_i2c_config_t config;
    stm32f4xx_i2c_fsm_t *fsm;

   /* Registers*/
    uint32_t i2c_cr1;
    :
```

```
    :
    :
    uint32_t i2c_fltr;
}STM32F4XXI2CState;
```

This structure is composed of several elements that fulfill various functions. The most important ones will be analyzed individually or mentioned as the section progresses. Although not all, some of the elements that make up this structure are:

- **Ten 32-bit registers** (`i2c_cr1` through `i2c_fltr`) tahat mirror the I2C registers in the STM32F4 reference manual.
- An `I2CBus *bus` pointer enabling connection to QEMU's generic I2C subsystem, allowing communication with any connected slave device model.
- Dual interrupt lines (`irq_event` and `irq_error`) that replicate hardware separation between event-driven and error-driven interrupt sources.

State Tracking Structure: `stm32f4xx_i2c_config_t`

A secondary structure, `config` (type `stm32f4xx_i2c_config_t`), tracks logic not directly visible in hardware registers (see Appendix F.1). This structure is essential due to the STM32F405's complex flag-clearing semantics. For example, the `ADDR` bit in `SR1` is cleared only when `SR1` is read *followed* by a read from `SR2`. The boolean member `flg_addr` records whether `SR1` has been read, allowing the subsequent read from `SR2` to correctly clear the `ADDR` bit.

Finite State Machine: Protocol Behavior

The sequential nature of the I2C protocol—where register accesses trigger state-dependent bus operations—necessitates a finite state machine (FSM) model. This FSM serves as the operational heart of the emulated I2C controller, translating register accesses into I2C bus transactions.

The implementation defines two FSM-related types:

- `stm32f4xx_i2c_fsm_trans_t`: A structure representing a single state transition.
- `stm32f4xx_i2c_fsm_t`: A structure containing a pointer to an array of transitions and the current FSM state.

The STM32F4XXI2CState structure incorporates a pointer to stm32f4xx_i2c_fsm_t. The FSM states, encoded as an enumeration, reflect the operational phases of the I2C controller in master mode as documented in the STM32F405 reference manual, including edge cases for 1, 2, and 3-byte reception where ACK/NACK timing differs.

code 5.3: fsm states encoded

```
/* stm32f4xx I2C fsm states */
typedef enum stm32f4xx_i2c_fsm_state_e
{
    STM32F4XX_I2C_DISABLED,
    STM32F4XX_I2C_IDLE,
    STM32F4XX_I2C_START,
    STM32F4XX_I2C_SENT_READ,
    STM32F4XX_I2C_SENT_WRITE,
    STM32F4XX_I2C_RECV_1BYTE,
    STM32F4XX_I2C_RECV_2BYTES_FIRST,
    STM32F4XX_I2C_RECV_2BYTES_SECOND,
    STM32F4XX_I2C_RECV_3BYTES,
    STM32F4XX_I2C_RECV_NBYTES,
    STM32F4XX_I2C_TRANSMITTING,
}stm32f4xx_i2c_fsm_state_t;
```

5.2.2 QEMU Object Model Integration

The STM32F4xx I²C controller emulation is integrated into QEMU's Object Model (QOM) through the following declarations:

code 5.4: QOM macros

```
#define TYPE_STM32F4XX_I2C "stm32f4xx-i2c"
OBJECT_DECLARE_SIMPLE_TYPE(STM32F4XXI2CState, STM32F4XX_I2C)
```

TYPE_STM32F4XX_I2C defines the unique QOM type identifier string "stm32f4xx-i2c", enabling device instantiation via qdev_new(TYPE_STM32F4XX_I2C) and object_initialize_child() calls throughout the SoC emulation. OBJECT_DECLARE_SIMPLE_TYPE establishes the type-safe relationship between the STM32F4XXI2CState instance structure and its QOM type, automatically generating casting macros.

Device Lifecycle

The first thing QEMU does is register the device within its kernel. This is done with the type_init macro. This macro is called before the main Qemu macro.

code 5.5: type_init macro

```
type_init(stm32f4xx_i2c_register_types)
```

The `TypeInfo` structure that represents the I2C controller is the following

code 5.6: `TypeInfo` structure

```
static const TypeInfo stm32f4xx_i2c_info =
{
    .name           = TYPE_STM32F4XX_I2C,
    .parent         = TYPE_SYS_BUS_DEVICE,
    .instance_size  = sizeof(STM32F4XXI2CState),
    .class_init     = stm32f4xx_i2c_class_init,
};
```

Upon device registration, QEMU invokes the `stm32f4xx_i2c_init()` function, Code 5.7 . This class initializer overrides the virtual methods of `SysBusDeviceClass` with STM32F4xx I2C-specific implementations, configures device properties via `device_class_set_props()`, assigns the realization callback `stm32f4xx_i2c_realize`, and registers the VM state description. In this specific implementation, the `stm32f4xx_i2c_init()` function it also allocates the FSM structure.

code 5.7: `stm32f4xx_i2c_init()`

```
static void stm32f4xx_i2c_class_init(ObjectClass *klass, void *data
    )
{
    DeviceClass *dc = DEVICE_CLASS(klass);

    device_class_set_props(dc, stm32f4xx_i2c_properties);
    dc->realize = stm32f4xx_i2c_realize;
    dc->vmsd    = &vmstate_stm32f4xx_i2c;
}
```

5.2.3 Memory-Mapped I/O Register Handling

Firmware executing on the emulated STM32F405 accesses the I2C controller through memory-mapped reads and writes to the APB1 peripheral space. For I2C1, this base address is `0x40005400`. QEMU intercepts these memory accesses via the registered `MemoryRegion` and dispatches them to register-specific handler functions.

Read Handler: `stm32f4xx_i2c_read()`

The `stm32f4xx_i2c_read()` function is launched when a register read operation is received, Code F.5. Within this function, a distinction is made between *simple storage registers* (e.g. `CR2`, `CCR`), which return their stored value directly, and *registers with side effects*, which trigger additional logic. For each of these registers with additional logic associated with its reading, an associated function has been created that implements said additional logic.

- `stm32f4xx_i2c_read_dr()` is executed for reading the `DR` register. It execute three critical operations:
  1. **Returns the received byte** from the internal buffer.
  2. **Clears flags** in `SR1`: specifically `RXNE` and `BTF`, signaling to firmware that the data has been read.
  3. **Triggers the FSM** via `stm32f4xx_i2c_fsm_fire()`, potentially fetching the next byte from the I2C bus and advancing the protocol state.
- `stm32f4xx_i2c_read_sr1()` is executed for reading the `SR1` register.
  - Reading `SR1` sets internal tracking flags (`flg_sb` and `flg_addr`). Per STM32F405 semantics, reading `SR1` is the *first step* in clearing `SB` (Start Bit) and `ADDR` (Address Match) flags. The actual clearing occurs only on subsequent reads of `DR` or `SR2`, as specified in the hardware reference manual.
- `stm32f4xx_i2c_read_sr2()` is executed for reading the `SR2` register.
  - If the tracking flag `flg_addr` is set, reading `SR2` clears the `ADDR` bit in `SR1`, completing the required "read SR1 then read SR2" sequence mandated by STM32F405 hardware.

Write Handler: `stm32f4xx_i2c_write()`

It is the same logic as the read handler but applied to writing registers. Control registers `CR1` and `CR2` require specialized handlers because writes can trigger significant state changes: starting I2C transactions (`START` bit), enabling interrupts, or resetting the peripheral. The data register write handler initiates actual transmission on the I2C bus. Configuration registers are updated directly in the state structure, Code F.4.

5.3 LIS3DH Accelerometer Implementation

The LIS3DH accelerometer emulation provides a complete I2C slave device implementation that faithfully reproduces the STMicroelectronics LIS3DH register map and configuration logic. The LIS3DH accelerometer device model follows the same fundamental which were explained for I2C controller. However, as an I2C slave device rather than a bus controller, the LIS3DH exhibits key architectural differences in its communication interface, device properties, and virtual machine state management. This section describes the implementation of the LIS3DH device model, focusing specifically on these critical distinctions.

5.3.1 Architectural Distinction: I²C Slave vs. SysBus Device

The fundamental difference between the STM32F4xx I2C controller and the LIS3DH accelerometer lies in their position within the system hierarchy:

- **STM32F4xx I2C Controller**: Inherits from `TYPE_SYS_BUS_DEVICE`, providing memory-mapped I/O regions that respond to CPU memory accesses at specific addresses. The controller acts as the **I2C bus master**, initiating transactions and managing the bus protocol.
- **LIS3DH Accelerometer**: Inherits from `TYPE_I2C_SLAVE`, implementing an I2C peripheral that responds to transactions initiated by the bus master. The LIS3DH has no MMIO interface accessible to the CPU; instead, it communicates exclusively through the I2C protocol using seven-bit addressing.

This difference means that different communication handlers are needed. While the I2C controller employs MMIO read/write functions (`stm32f4xx_i2c_read()` and `stm32f4xx_i2c_write()`), the LIS3DH implements **I2C protocol callbacks** (`lis3dh_i2c_event()`, `lis3dh_i2c_send()`, and `lis3dh_i2c_recv()`).

5.3.2 Device Structure and Configuration State

The LIS3DH device state structure (`LIS3DHState`), (Code F.6), is defined in the code file `lis3dh.h`. Similar to the `STM32F4XXI2CState` structure, the `LIS3DHState` structure uses variables of type `uint8_t` to represent the actual hardware registers.

The `LIS3DHState` structure also implements two additional structures:

- **The** `lis3dh_config_t` structure, which caches the values of the decoded registers (operating mode, full range, output data rate) to avoid repeated extraction of bit fields.

- **The** `lis3dh_params_t` structure contains the calculated values (sensitivity coefficients, bit shift values) derived from the values in the `lis3dh_config_t` structure, thus optimizing data conversion.

As mentioned in Chapter 3.1.2, the LIS3DH accelerometer has two programmable interrupt output pins: INT1 and INT2. Both pins have their respective configuration registers. To use these interrupt lines with other emulated devices, these two pins are represented in the `LIS3DHState` structure as `qemu_irq` signals and, during the execution of the `lis3dh_realize()` function, are initialized as GPIO elements.

5.3.3 I2C Slave Interface Implementation

As an I2C slave device, the LIS3DH implements the `I2CSlaveClass` virtual methods: `event()`, `recv()`, and `send()`. The event() method is the event handler, it is responsible for responding to events coming from the I2C bus (START, STOP, END). In the current implementation of the LIS3DH accelerometer it has been defined with the function lis3dh_i2c_event(), Code F.7.

The lis3dh_i2c_recv() function is called every time the I2C master reads a byte from the slave. The function retrieves the value of the register at the address of the current pointer via lis3dh_read_register() and returns it to the master. If auto-increment was previously enabled during subaddressing, the record pointer is subsequently incremented, allowing burst reads of consecutive records. This design ensures that the firmware can read all six bytes of throttle output (OUT_X_L/H, OUT_Y_L/H, OUT_Z_L/H) in a single I2C read transaction.

The lis3dh_i2c_send() function handles all bytes transmitted by the I2C master to the LIS3DH. It implements a two-phase protocol: upon receiving the first byte after the I2C START condition, the function extracts the register address from bits [6:0] and the automatic increment control bit, putting the device into data writing mode. Subsequent transmitted bytes are written to the current register address via lis3dh_write_register(), and if auto-increment is enabled, the register pointer automatically advances to the next address. This mechanism allows the firmware to set multiple consecutive control registers in a single I2C transaction without reissuing the subaddress byte, which matches the behavior of LIS3DH physical hardware.

5.3.4 Register Map and Configuration Logic

The LIS3DH accelerometer register map implementation comprises 64 registers, including those for identification, control, status, and output data. As explained in Section 4.3, the accelerometer model was designed with the idea that whenever the firmware modifies a control register, the emulator immediately recalculates all derived operating parameters. This ensures that the device state remains consistent

and that subsequent firmware operations use correct parameter values, mirroring the atomic configuration behavior of the physical hardware. For example, when the firmware writes to `CTRL_REG1` to change the output data rate or to `CTRL_REG4` to modify the full-scale range, an associated function is immediately invoked that updates the corresponding parameters; in this case, the sensitivity coefficients are recalculated and the bit offsets are updated.

5.3.5 QOM Property Integration

The `lis3dh_initfn()` function is the QOM instance initializer, invoked during device object creation. It registers four dynamic properties (`accel-x`, `accel-y`, `accel-z`, and `temp`) that expose the sensor measurement state to the host emulation environment. Each property is bound to a pair of getter/setter callbacks: writing to `accel-x` invokes `lis3dh_set_accel_x()`, which converts the input acceleration value (in milligravity) to the sensor's raw format, updates internal output registers, and triggers data-ready interrupts, exactly as specified in the Section 4.3; reading from `accel-x` invokes `lis3dh_get_accel_x()`, which retrieves and scales the stored acceleration data. This implementation ensures bit-accurate output register values matching the datasheet specifications, enabling firmware using the STM HAL library to read acceleration data without modification.

5.4 STM32F405 SoC Integration

The STM32F405 SoC integration extends QEMU's existing `stm32f405_soc` device to instantiate and configure the three I2C controllers (I2C1, I2C2, and I2C3) defined in the STM32F405 reference manual. This section details the structural modifications, instance initialization, and bus creation required to integrate the I2C subsystem into the SoC model.

5.4.1 Device State Structure Extension

By copying the way other elements have been implemented in the SoC structure, and having previously defined the constant `STM_NUM_I2CS` with a value of 3—due to the three I2C peripherals available in the STM32F405 microcontroller—, an array of type STM32F4XXI2CState, of length `STM_NUM_I2CS`, has been implemented in the structure, whose elements represent the three I2C controllers , Code 5.2.

```
typedef struct STM32F405State {
    SysBusDevice parent_obj;

    /* ARM Cortex-M4 CPU */
    ARMv7MState armv7m;

    /* Existing peripherals */
    :
    STM32GPIOState gpio[STM_NUM_GPIOS];
    STM32F4xxSPIState spi[STM_NUM_SPIS];
    :
    STM32F4XXI2CState i2c[STM_NUM_I2CS];

    /* Other elements */
    :
} STM32F405State;
```

5.4.2 Instance Initialization

The `stm32f405_soc_initfn()` function, invoked during device instantiation by QEMU's QOM framework, initializes each I2C controller instance and establishes parent-child relationships within the object hierarchy. The relevant code that has been added is:

code 5.8: I2C instance initialization in SoC initfn

```
static void stm32f405_soc_initfn(Object *obj)
{
    STM32F405State *s = STM32F405_SOC(obj);
```

```
    /* ... other peripheral initializations ... */

    /* Initialize each I2C controller */
    for (i = 0; i < STM_NUM_I2CS; i++)
    {
        object_initialize_child(obj, "i2c[*]", &s->i2c[i],
           TYPE_STM32F4XX_I2C);
    };

    /* ... other peripheral initializations ... */
}
```

The `object_initialize_child()` macro is another QOM utility. It is executed before the property configuration and realization, and that performs the following tasks:

1. Allocates and initializes the child object (`STM32F4xxI2CState`).
2. Establishes a parent-child relationship in the object tree for proper lifecycle management.
3. Assigns a canonical path in QOM (e.g., `/machine/soc/i2c[0]`).

5.4.3 Device Realization and Memory Mapping

The I2C controller's register blocks are mapped to the ARM memory address space at the addresses specified in the STM32F405 datasheet. These addresses define where the firmware accesses the control and status registers of the I2C peripherals. In QEMU's code, these addresses are represented as the next arrays:

code 5.9: I2C base address mapping

```
static const uint32_t i2c_addr[] = {
    0x40005400,  /* I2C1 base address */
    0x40005800,  /* I2C2 base address */
    0x40005C00   /* I2C3 base address */
};
```

The STM32F405 NVIC (Nested Vectored Interrupt Controller) supports 96 interrupt sources (IRQ 0–95), each connected to specific hardware peripherals and events. The I2C controllers generate two distinct interrupt types: *event interrupts* (EV) signal normal I2C protocol events (START condition detected, address matched, data byte transmitted), while *error interrupts* (ER) signal protocol violations (bus error, arbitration loss, acknowledge failure). This is why in QEMU the I2C interrupt array is organized as interrupt pairs for each controller (event, error):

code 5.10: I2C interrupt vector numbers

```
static const int i2c_irq[] = {
    31, 32,     /* I2C1: event (IRQ 31), error (IRQ 32) */
    33, 34,     /* I2C2: event (IRQ 33), error (IRQ 34) */
    72, 73      /* I2C3: event (IRQ 72), error (IRQ 73) */
};
```

The `stm32f405_soc_realize()` function completes the I2C integration by realizing each controller device, retrieving the public bus reference, mapping I2C registers into the SoC's memory space, and connecting interrupt lines to the NVIC.

code 5.11: I2C realization and memory mapping

```
static void stm32f405_soc_realize(DeviceState *dev_soc, Error **
    errp)
{
    STM32F405State *s = STM32F405_SOC(dev_soc);
    MemoryRegion *system_memory = get_system_memory();
    DeviceState *dev, *armv7m;
    DeviceState *lis3dh;;
    SysBusDevice *busdev;
    Error *err = NULL;
    int i;

    /* I2C devices */
    for (i = 0; i < STM_NUM_I2CS; i++)
    {
        dev = DEVICE(&s->i2c[i]);

        char *bus_name = g_strdup_printf("i2c%d", i+1);
        qdev_prop_set_string(dev, "bus-name", bus_name );
        g_free(bus_name);

        if (!sysbus_realize(SYS_BUS_DEVICE(&s->i2c[i]), errp))
            return;

        busdev = SYS_BUS_DEVICE(dev);
        sysbus_mmio_map(busdev, 0, i2c_addr[i]);

        /* Connect the two interrupts (event and error) for each
           I2C controller */
        sysbus_connect_irq(busdev, 0,
                           qdev_get_gpio_in(armv7m, i2c_irq[i*2]));
        sysbus_connect_irq(busdev, 1,
                           qdev_get_gpio_in(armv7m, i2c_irq[i*2+1])
                               );
    }

    /* ... other peripheral realizations ... */
}
```

5.5 Integration and Machine Configuration

Once the I²C controllers were implemented on the STM32F405 SoC and the LIS3DH accelerometer model was included in QEMU, the final integration step was to establish an I²C connection between an instance of the accelerometer and an instance of the STM microcontroller.

As explained in Chapter 4, the Netduino Plus 2 board already has a model implemented in QEMU that uses an STM32F405 microcontroller. For this reason, it was decided to integrate a LIS3DH accelerometer as a component of the Netduino Plus 2 board itself, thus creating a complete integrated development environment. To perform functional tests and verify the interrupt operation, the INT1 interrupt pin of the accelerometer was wired to the PA1 pin of the STM32F405 microcontroller.

This section details the device hierarchy, interrupt pin routing, and initialization sequence that connects the LIS3DH accelerometer model to the STM32F405 SoC infrastructure.

5.5.1 Netduino Plus 2 Initialization & Pin Connection Architecture

The `netduinoplus2_init()` (Code 5.12) function is responsible for the complete system assembly, following QEMU's device instantiation hierarchy. The first thing this function does is instantiate the STM32F405 SoC using the `qdev_new(TYPE_STM32F405_SOC)` function and realize it with the `sysbus_realize_and_unref()` function. This last function activates the SoC's internal `stm32f405_soc_realize()` function, which instantiates the three I²C controllers and populates the `s->I²C[0].bus` pointer with the I²C1 bus reference, making it available for connecting external devices.

After the SoC is realized, the LIS3DH accelerometer is created with `qdev_new(TYPE_LIS3DH)`, configured with its 7-bit I²C address `0x18` (pin SA0 grounded), and connected to I²C1 using `I2C_slave_realize_and_unref()`.

The LIS3DH's INT1 interrupt output is routed to the STM32's EXTI 1 line using `qdev_connect_gpio_out()`, allowing the firmware to receive data-ready notifications. Finally, `armv7m_load_kernel()` loads the user-supplied firmware binary into the emulated flash memory, resulting in a fully functional Netduino Plus 2 emulation environment.

code 5.12: Netduino Plus 2 machine initialization

```
static void netduinoplus2_init(MachineState *machine)
{
    DeviceState *dev;
    Clock *sysclk;

    /* This clock doesn't need migration because it is fixed-
       frequency */
    sysclk = clock_new(OBJECT(machine), "SYSCLK");
    clock_set_hz(sysclk, SYSCLK_FRQ);

    /* Instantiate STM32F405 SoC */
    dev = qdev_new(TYPE_STM32F405_SOC);
    object_property_add_child(OBJECT(machine), "soc", OBJECT(dev));
    qdev_connect_clock_in(dev, "sysclk", sysclk);
    sysbus_realize_and_unref(SYS_BUS_DEVICE(dev), &error_fatal);

    /* Initialize LIS3DH on I$^2$C1 */
    STM32F405State *s = STM32F405_SOC(dev);
    DeviceState *lis3dh = qdev_new(TYPE_LIS3DH); // Creating LIS3DH
        accelerometer

    I$^2$C_slave_set_address(I$^2$C_SLAVE(lis3dh), 0x18); //
       Setting I$^2$C address to 0x18
    I$^2$C_slave_realize_and_unref(I$^2$C_SLAVE(lis3dh),
                    s->I$^2$C[0].bus, &error_fatal);    //
                       Connecting to I$^2$C1 bus

    qdev_connect_gpio_out(lis3dh, 0, qdev_get_gpio_in(DEVICE(&s->
       exti), 1));

    /* Load firmware into flash memory */
    armv7m_load_kernel(ARM_CPU(first_cpu),
                       machine->kernel_filename,
                       0, FLASH_SIZE);
}
```

5.5.2 QEMU Device Hierarchy

When a Netduino Plus 2 machine instance is created, a hierarchical device tree is also created where the LIS3DH exists as an I^2C slave device connected to the I^2C1 bus of the STM32F405 SoC. Figure 5.5 illustrates the complete device hierarchy:

```
netduinoplus2 (Machine)
\---- STM32F405_SOC
    |---- I$^2$C[0] (I$^2$C1 Bus)
    |   \---- lis3dh (I$^2$C Slave @ 0x18)
    |       |---- int1 (qemu_irq out)  ----\
    |       \---- int2 (qemu_irq out)      |
    |                                       |
    \---- exti (STM32F4XX_EXTI)             |
        |---- gpio_in[0] <- (EXTI0)         |
        |---- gpio_in[1] <- (EXTI1) <-------/
        |---- gpio_in[2] <- (EXTI2)
        \---- ...
        \---- irq_out[7] -> NVIC IRQ 7 (EXTI1_IRQn)
```

Figure 5.5: qemu hierarchy

This hierarchy demonstrates the signal flow from the LIS3DH interrupt outputs through the EXTI (External Interrupt) controller to the NVIC (Nested Vectored Interrupt Controller), which ultimately triggers the ARM Cortex-M4 core's interrupt service routine.

5.6 Firmware

With all hardware components modeled and integrated into the QEMU emulator, only the firmware for loading onto the Netduino Plus 2 board and controlling the LIS3DH accelerometer remains. Although Seeed-Studio provides an open-source library for managing the LIS3DH accelerometer[1], this library has the disadvantage of being designed for Arduino or Raspberry Pi boards, not for STM32-based platforms. Therefore, based on the Seeed-Studio library, a custom C library was developed to manage the LIS3DH accelerometer.

This library enables device configuration and reading of acceleration and temperature data through the I^2C bus. It is designed for use with STM32F4 series microcontrollers, specifically the STM32F405, utilizing STMicroelectronics' Hardware Abstraction Layer (HAL) as its foundation. The library comprises three main files:

1. `lis3dh_types.h`: Contains data type definitions, enumerations, and structures necessary for the sensor interface.
2. `lis3dh_driver.h`: Defines constants, macros, and public function prototypes for the driver.
3. `lis3dh_driver.c`: Implements the configuration and communication logic with the LIS3DH sensor.

5.6.1 Data Structures

To encapsulate accelerometer information, the following structures were defined in `lis3dh_types.h`:

- `lis3dh_config_t`: Stores device configuration, including measurement scale (2g, 4g, 8g, 16g), operating mode (high resolution, normal, low power), output data rate (ODR), and enablement of temperature sensor and high-pass filter.
- `lis3dh_data_t`: Contains sensor readings, i.e., accelerations on three axes (in mg) and temperature (in degrees Celsius).
- `lis3dh_t`: Main structure grouping the I^2C handler, device address, configuration, sensitivity, and data.

[1] https://github.com/Seeed-Studio/Seeed_Arduino_LIS3DHTR

5.6.2 Implemented Functionality

The library provides the following functions:

Initialization and Configuration

- `lis3dh_init`: Initializes the accelerometer structure, verifies I^2C communication, and configures default or user-provided parameters.
- `lis3dh_configuration`: Applies specific device configuration, setting operating mode, scale, sampling rate, and enabling axes.

Interrupt Configuration

- `lis3dh_enable_drdy_interrupt`: Enables Data-Ready (DRDY) interrupt on the accelerometer's INT1 pin, notifying the microcontroller when new data is available.
- `lis3dh_read_int_source`: Reads the interrupt source register, which also clears the interrupt.

Data Reading

- `lis3dh_read_acc`: Reads acceleration registers, converts raw values to milligravity (mg) units, and stores them in the data structure.
- `lis3dh_read_temp`: Reads the temperature register and converts the value to degrees Celsius.

Low-Level Functions

Four low-level functions have been designed in total. The functions `lis3dh_read_reg` and `lis3dh_write_reg` read and write a single accelerometer register, while `lis3dh_read_regs_array` and `lis3dh_write_regs_array` read and write multiple consecutive registers. These four functions are essentially wrappers for the HAL functions `HAL_I2C_Mem_Write` and `HAL_I2C_Mem_Read`. These four functions are the ones that the functions explained above use to access the LIS3DH registers via the I2C protocol

Chapter 6

Testing

The validation of the emulated STM32F405 I^2C controller and LIS3DH accelerometer required a comprehensive testing strategy that verifies functional correctness, protocol compliance, and firmware portability. Unlike unit tests that isolate individual functions, the testing approach adopted in this thesis employs full-system integration testing: unmodified STM32CubeF4 HAL-based firmware executing within QEMU, interacting with the emulated peripherals via the emulated I^2C bus, and producing observable outputs via UART console redirection.

Testing objectives:

1. **Protocol Fidelity:** Verify that I^2C subaddressing, auto-increment, and register access sequences match the LIS3DH datasheet specifications.
2. **Configuration Correctness:** Confirm that writing to control registers (CTRL_REG1 to CTRL_REG6) produces correct internal state transitions (operating mode, full-scale range, output data rate).
3. **Data Acquisition:** Validate that acceleration values injected via QEMU's QOM properties propagate correctly through register reads and produce accurate scaled output in milligravity units.
4. **Interrupt Behavior:** Test Data-Ready (DRDY) interrupt generation on INT1, GPIO EXTI handling, and latched/non-latched interrupt modes.
5. **HAL Driver Compatibility:** Demonstrate that STMicroelectronics' reference HAL drivers (`HAL_I2C_Mem_Read()`, `HAL_I2C_Mem_Write()`) execute without modification.

6.1 STM32CubeIDE Configuration

As mentioned in the section 2.3.1, all firmware development for this project was performed using `STM32CubeIDE` and its associated tools. This section describes the pre-configuration carried out in `STM32CubeIDE` prior to defining the functional tests.

`STM32CubeIDE` allows the creation of projects specifically targeted to particular microcontrollers or development boards. Using this feature, a new project is created using a preconfigured STM32F4-based evaluation board as the base. This board uses the STM32F407 microcontroller, which, as previously noted, is compatible with STM32F405 configurations.

Once the project is created, `STM32CubeMX` opens automatically, allowing the microcontroller to be configured visually as follows:

1. Enable I2C1 and configure it for 100 kHz operation with 7-bit addressing. Then assign the I2C1_SCL and I2C1_SDA lines to pins PB6 and PB7, respectively.
2. Configure a UART peripheral to enable debugging.
3. Reconfigure pin PA1, which is wired to the LIS3DH INT1 pin, to **GPIO_EXTI1** mode. In the configuration for this pin, select "External interrupt mode with rising edge trigger detection" and "No pull-up and no pull-down". Then, in the **NVIC** tab, enable the **EXTI line 1 interrupt** and assign it a priority of 5.

Once these configurations are applied and saved, `CubeMX` generates the code required to initialize the I^2C, GPIO, and EXTI lines. Within this generated code, and before integrating the LIS3DH library or defining the tests, the following two modifications must be made:

4. In the `main()` function, locate and comment out the `SystemClock_Config()` call.
5. Define the `HAL_GPIO_EXTI_Callback()` function, which is executed automatically when the DRDY interrupt is triggered on INT1.

The `SystemClock_Config()` function is commented out because the STM32F405 model implemented in QEMU does not accurately emulate the entire clock subsystem. As a result, some function calls within `SystemClock_Config()` return `HAL_ERROR`, which in turn causes `Error_Handler()` to be invoked and enter its infinite loop.

6.2 Test Cases

This section presents a series of tests and their results which, in the developer's opinion, cover all aspects necessary to validate the operation of the project, both of the I2C controller and the LIS3DH.

6.2.1 Test Cases

Test Case 1: Device Initialization and WHO_AM_I Verification

Objectives: Verify that the LIS3DH device responds to I²C transactions and returns the correct device ID.

Procedure:

1. Initialize LIS3DH in Normal mode, ±4g, 100 Hz.
2. Read WHO_AM_I register (0x0F).
3. **Expected:** `0x33` (LIS3DH device ID, datasheet §8.1).

Code:

code 6.1: WHO_AM_I verification test case

```
static void testcase1() {
    printf("---␣Test␣Case␣1:␣WHO_AM_I␣Verification␣---\n");
    lis3dh_config.fscale = LIS3DH_FS_4G;
    lis3dh_config.mode = LIS3DH_MODE_NORMAL;
    // ... init ...
    HAL_StatusTypeDef status = lis3dh_init(&lis3dh_acc, &hi2c1,
                                           LIS3DH_I2C_DEFAULT_ADDRESS
                                               , &lis3dh_config);
    if (status == HAL_OK)
        printf("PASS:␣LIS3DH␣init␣OK,␣WHO_AM_I␣=␣0x33\n");
    else
        printf("FAIL:␣LIS3DH␣init␣failed\n");
}
```

Test Case 2: Operating Mode Configuration

Objective: Validate that CTRL_REG1/CTRL_REG4 correctly configure all operating modes.

Procedure: For each mode (Normal/High-Res/Low-Power): configure → read → verify.

Code:

code 6.2: Operating mode configuration test

```
static bool testcase21() {  // Normal mode
    lis3dh_set_mode(&lis3dh_acc, LIS3DH_MODE_NORMAL);
    lis3dh_mode_t mode = lis3dh_get_mode(&lis3dh_acc);
    return (mode == LIS3DH_MODE_NORMAL);
}
// Similar tests for HIGH_RES and LOW_POWER
```

Test Case 3: Full-Scale Configuration

Objective: Validate configuration of ±2g/±4g/±8g/±16g ranges in CTRL_REG4.

Procedure: For each full-scale range: configure → read → verify.

Code:

code 6.3: Full-scale range configuration test

```
static bool testcase31() {
    lis3dh_set_full_scale(&lis3dh_acc, LIS3DH_FS_2G);
    lis3dh_fscale_t fs = lis3dh_get_full_scale(&lis3dh_acc);
    return (fs == LIS3DH_FS_2G);
}
// Similar tests for +-4g/+-8g/+-16g
```

Test Case 4: Acceleration Data Injection and Conversion

Objective: Verify complete data flow: QOM injection → registers → I^2C read → firmware scaling.

Test sequence:

1. Configure Normal mode, ±4g (So = 8 mg/LSB).
2. Inject via QEMU: `qom-set accel-x 1000`.
3. Firmware reads via `lis3dh_read_acc()` → verifies 1000 mg.

Code:

code 6.4: Acceleration data injection test

```
lis3dh_config.fscale = LIS3DH_FS_4G;
lis3dh_config.mode = LIS3DH_MODE_NORMAL;
// ... config & enable DRDY ...
// QEMU: qom-set accel-x 1000
lis3dh_read_acc(&lis3dh_acc);
printf("INT! X: %.2f mg", lis3dh_acc.data->acc_mg[0]);  // Expected
    : 1000.00
```

Test Case 5: Data-Ready Interrupt Generation

Objective: Validate INT1 interrupt generation when new data is available.

Configuration: `lis3dh_enable_drdy_interrupt()`.

Test sequence:

1. Inject X/Y/Z data via QOM properties.
2. Verify EXTI1 (PA1) triggers → `HAL_GPIO_EXTI_Callback()`.
3. Read data + INT1_SRC (clears latched interrupt).
4. INT1 returns low.

Code:

code 6.5: Data-Ready interrupt handling

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == LIS3DH_INT1_Pin) g_data_ready = true;
}
// Main loop
if (g_data_ready)
{
  g_data_ready = false;  // Clear flag

  // Read acceleration data
  status = lis3dh_read_acc(&lis3dh_acc);
  if (status == HAL_OK)
  {
    printf("INT! X: %.2f mg, Y: %.2f mg, Z: %.2f mg\n",
        lis3dh_acc.data->acc_mg[0],
        lis3dh_acc.data->acc_mg[1],
        lis3dh_acc.data->acc_mg[2]);
  }
  else
  {
    printf("STM32: Read acc ERROR after interrupt\n");
```

```
  }

  // Read INT1_SRC to verify/clear interrupt source
  uint8_t int_src = 0;
  lis3dh_read_int_source(&lis3dh_acc, &int_src);
  printf("INT1_SRC: 0x%02X\n", int_src);
}
```

6.3 Results

After executing QEMU loaded with the test firmware using the command shown in Figure 6.6:

code 6.6: QEMU command for automated LIS3DH testing

```
sudo ./qemu-system-arm -M netduinoplus2 -kernel firmware_test_4.elf
    -serial stdio
```

the following output confirms that the system fully meets all five testing objectives established at the beginning of the chapter:

code 6.7: Automated test execution output

```
=== LIS3DH EMULATION TEST SUITE ===

--- Test Case 1: WHO_AM_I Verification ---
 PASS: LIS3DH init OK, WHO_AM_I = 0x33

--- Test Case 2: Operating Mode Configuration ---

 Test Case 2.1: LIS3DH_MODE_NORMAL
         TEST PASSED
 Test Case 2.2: LIS3DH_MODE_HIGH_RES
[LIS3DH] Mode: high
[LIS3DH] acc sensitivity: 1.0
         TEST PASSED
 Test Case 2.3: LIS3DH_MODE_LOW_POWER
[LIS3DH] Mode: low
[LIS3DH] acc sensitivity: 16.0
[LIS3DH] Mode: low
[LIS3DH] acc sensitivity: 16.0
         TEST PASSED

 PASSED:  Operating Mode Configuration Test case

--- Test Case 3: Full-Scale Configuration ---
 Test Case 3.1: LIS3DH_FS_2G
```

```
PASS
 Test Case 3.2: LIS3DH_FS_4G
[LIS3DH] FS: LIS3DH_FS_4G
[LIS3DH] acc sensitivity: 32.0
PASS
 Test Case 3.3: LIS3DH_FS_8G
[LIS3DH] FS: LIS3DH_FS_8G
[LIS3DH] acc sensitivity: 64.0
PASS
 Test Case 3.4: LIS3DH_FS_16G
[LIS3DH] FS: LIS3DH_FS_16G
[LIS3DH] acc sensitivity: 192.0
PASS

 PASS: Full-scale Test case

--- Test Case 4 & 5: Full-Scale Configuration ---
[LIS3DH] Mode: normal
[LIS3DH] acc sensitivity: 48.0
[LIS3DH] FS: LIS3DH_FS_4G
[LIS3DH] acc sensitivity: 8.0
[LIS3DH] ODR: LIS3DH_ODR_100

STM32: LIS3DH initialized with DRDY interrupt on INT1
LIS3DH: Input: 1000.0 mg -> 125.0 LSB -> 125 raw

LIS3DH: out_x_h: 0x1f, out_x_l: 0x40
LIS3DH: Input: 1000.0 mg -> 125.0 LSB -> 125 raw

LIS3DH: out_y_h: 0x1f, out_y_l: 0x40
LIS3DH: Input: 1000.0 mg -> 125.0 LSB -> 125 raw

LIS3DH: out_z_h: 0x1f, out_z_l: 0x40
LIS3DH: INT1 pulsed (DRDY, non-latched)
Calculated So = 8.000 mg/LSB (mode=0, fscale=0)
INT! X: 1000.00 mg, Y: 1000.00 mg, Z: 1000.00 mg
INT1_SRC: 0x00
```

6.4 Objective-Result Mapping

Table 6.1: Test Objective to Result Mapping

Objective	**Test Case**	**Result**	**Justification**
Protocol Fidelity	Test Case 1	PASSED	I^2C subaddressing; auto-increment correct
Configuration Correctness	Test Case 2 & 3	PASSED	Write in the control registers change the internal logic
Data Acquisition	Test Case 4	PASSED	QOM → registers → perfect mg scaling
Interrupt Behavior	Test Case 5	PASSED	DRDY → EXTI → clear non-latched working
HAL Compatibility	All (Native)	PASSED	`HAL_I2C_MemRead /Write()` unmodified

These tests, though simple in description, together with additional undocumented validations, verify the complete functionality of the system developed in this thesis:

- QEMU LIS3DH emulator with QOM injection and DRDY interrupt.
- STM32F405 I^2C controller with datasheet register fidelity and native HAL support.
- Interrupt-driven firmware driver, portable and reusable.
- End-to-end validation from QOM properties to scaled mg output.

The results obtained demonstrate that the emulated system behaves identically to its physical counterpart, meeting all established objectives and fully validating the technical correctness of the work performed.

Chapter 7

Conclusions and Future Work

This thesis has successfully achieved the six specific objectives defined in Chapter 1, making a significant contribution to the MICROLAB and SIVALSE projects through the complete emulation of the I^2C protocol on the STM32F405 microcontroller and LIS3DH accelerometer within the QEMU framework. Table 7.1 summarizes the achievement of each objective with quantitative metrics extracted from the developed artifacts.

The most significant technical contribution is the complete implementation of the STM32F405 I^2C controller, which faithfully reproduces the register-level behavior of the peripheral according to Its Reference Manual. This model implements a finite state machine (FSM) with 12 states that translates firmware register write/read sequences (CR1.START, CR1.ACK, DR writes, etc.) into QEMU I^2C bus events, supporting 7-bit master mode, EV/ER interrupts, and full compatibility with the STM32CubeIDE HAL library.

Additionally, a complete LIS3DH accelerometer model was developed as an I^2C slave device that reproduces the 64-register map from the datasheet, supports all three operating modes (Normal/High-Res/ Low-Power), generates DRDY interrupts on INT1, and exposes QOM properties (`accel-x/y/z`, `temp`) for controlled test data injection. Integration into the `netduinoplus2` machine enables execution of real STM32 firmware that configures the LIS3DH via I^2C1 (address 0x18), reads acceleration data, and responds to EXTI1 interrupts, thus validating end-to-end HAL compatibility.

The results demonstrate that the emulated system is functionally equivalent to the physical hardware for MICROLAB educational use cases: I^2C initialization, control register writes (CTRL_REG1-6), multi-byte reads with auto-increment (OUT_X/Y/Z_L/H), and data-ready interrupt handling. This contribution enables virtual I^2C laboratories without physical hardware, facilitating remote teaching and self-paced learning in Embedded Systems.

Objective	**Achievement Status**	**Key Contributions**
Analyze STM32F405 and LIS3DH architecture focusing on I^2C protocol	Fully Achieved	Comprehensive analysis in Chapter 6 with datasheet breakdowns, register maps, protocol timing diagrams.
Develop STM32F405 I^2C controller emulation in QEMU	Fully Achieved	Complete implementation. FSM with 11 states, full register support, EV/ER interrupts. Compatible with STM32 HAL library.
Implement LIS3DH accelerometer as I^2C slave in QEMU	Fully Achieved	Complete model. 64-register map, 3 operating modes, QOM properties for accel injection, DRDY interrupt generation.
Integrate models into unified virtual system executing real STM32 firmware	Fully Achieved	Extended `netduinoplus2` machine. SoC integration with correct memory mapping and NVIC routing.
Validate using STM32 HAL firmware in STM32CubeIDE	Fully Achieved	Validated with HAL I^2C driver: init, multi-byte R/W, auto-increment, interrupt handling. Test cases passed: register config, acceleration reads, DRDY interrupts.

Table 7.1: Project Objectives Achievement Summary

7.1 Future Work

Although the objectives have been met, clear extension opportunities exist that would enhance MICROLAB's educational impact:

- **I²C Controller Expansion**: Implement slave mode, 10-bit addressing, timing registers (CCR, TRISE), DMA transfers, and SMBus/PMBus to support advanced UPM laboratory cases.
- **LIS3DH Model Improvements**: Add FIFO buffer (32 levels), click/tap detection (CLICK_CFG), threshold interrupts (INT1_CFG), complete temperature sensor, and auxiliary ADC channels, achieving 100% datasheet coverage.
- **Automated Testing Framework**: Develop QTest test cases to validate HAL sequences (init, read/write, error conditions) and edge cases (NACK handling, bus arbitration), along with STM32CubeIDE validation firmware for continuous regression testing.
- **MICROLAB Visualizer Integration**: Expose I²C bus state (SCL/SDA timing, START/STOP/DATA transactions) and LIS3DH registers via WebSocket for real-time visualization in the React interface.
- **Scalability to Other Peripherals**: Apply this methodology to emulate UART/USART, SPI, and ADC on the STM32F405, creating a complete virtual laboratory for the STM32F4-DISCOVERY board compatible with all Embedded Systems laboratory exercises.

These extensions would maintain the educational focus, prioritizing HAL firmware compatibility and observability for teaching purposes.

Appendix A

Ethical, Economic, Social, and Environmental Aspects

A1. Introduction

The project consists of QEMU emulation of the I^2C protocol for the STM32F405 microcontroller and LIS3DH accelerometer, aimed at contributing to the UPM's virtual MICROLAB laboratory for Embedded Systems education. This development addresses the educational need to simulate STM32F4-DISCOVERY boards without physical hardware, reducing economic and logistical barriers in remote and self-paced teaching. The relevant aspects identified are: **social** (democratized access to laboratories), **economic** (hardware cost reduction), **ethical** (open-source licenses and privacy in virtual education), and **environmental** (elimination of electronic waste). The main stakeholders are UPM students/teachers, the SIVALSE/MICROLAB project, and the QEMU open-source community.

A2. Description of Relevant Impacts Related to the Project

Among the impacts identified in the analysis phase, the most relevant stand out as the **positive social impact**: democratizes access to I^2C and sensorization practices for ∼200 annual Embedded Systems students at UPM, eliminating dependence on physical boards (∼20€/unit) and enabling inclusive/remote teaching. The **positive environmental impact**: avoids acquisition of ∼500 STM32/LIS3DH boards over 2-3 years (∼10,000€), promoting open-source software reuse. Economically, it reduces laboratory operating costs (∼5,000€/year in hardware/replacements). Minor negative impacts include host energy consumption (∼50W/h during use) and GPL/MIT license dependency. Prioritized stakeholders are students (direct beneficiaries), teachers (users), SIVALSE (maintainers), and STMicroelectronics (emulated hardware IP). Ethical negative impacts were discarded due to non-commercial educational use and technical fidelity without proprietary reverse-

engineering.

A3. Detailed Analysis of One of the Impacts

The **social impact: Democratization of educational access** is analyzed in detail. *Quantification*: At UPM, ∼200 students/year perform I^2C practices with STM32F4-DISCOVERY + LIS3DH (∼25€/kit). Virtual MICROLAB eliminates this cost, enabling unlimited executions on any PC. *Qualitative analysis*: Reduces digital divide (students without hardware access), supports 24/7 self-paced learning, and enables hybrid teaching post-COVID. *Mitigated risks*: HAL firmware verification ensures functional equivalence. *Success indicators*: STM32CubeIDE compatibility, integration in Netduino Plus 2 (QEMU machine), and potential for 10+ new practices (sensorization, digital filters). Compared to commercial alternatives, QEMU being open-source maximizes social impact without economic barriers.

A4. Conclusions

Integrating sustainability criteria has added significant value to the project, transforming a technical emulation into a tool with measurable social and environmental impact. Ethically, using open-source licenses (GPL/MIT) and ST datasheet fidelity promotes transparency and academic reuse. Socially, it democratizes embedded education for hundreds of UPM students. Economically, it saves thousands of euros in hardware. Environmentally, it avoids electronic waste and unnecessary energy consumption. These criteria guided key decisions (prioritizing HAL-compatibility over host optimization, documenting limitations), enhancing MICROLAB's pedagogical utility. The project exemplifies responsible engineering: sustainable software that multiplies educational access without compromising technical quality or generating negative impacts.

Appendix B

Budget

This annex contains the necessary information on the development cost of this project.

LABOR COST (direct cost)	**Hours**	**Price/hour**	**Total**
	450	15 euro	**6,750 euro**

COST OF MATERIAL RESOURCES (direct cost)	**Purchase price**	**Use in months**	**Depreciation (in years)**	**Total**
Ordenador personal (Software incluido).....	1.500,00 €	6	5	150,00 €
STM32F407-DIC1	20,00 €	6	0	20,00 €

TOTAL CODT OF MATERIAL RESOURCESS	**170,00 €**

GENERAL EXPENSES (indirect cost)	15 %	sobre CD	**1038,00 €**
INDUSTRIAL PROFIT	6 %	sobre CD + CI	**405,00 €**

SUBTOTAL BUDGET		**8.193,00 €**
APPLICABLE IVA	21 %	**1.720,53 €**

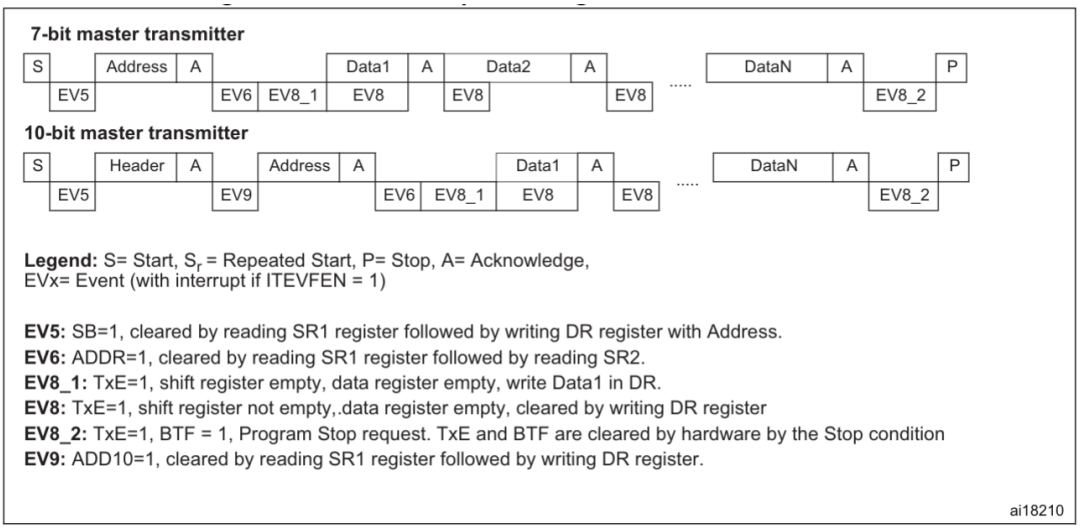
TOTAL BUDGET	**9.913,53 €**

Table B.1: Economic Budget

Appendix C

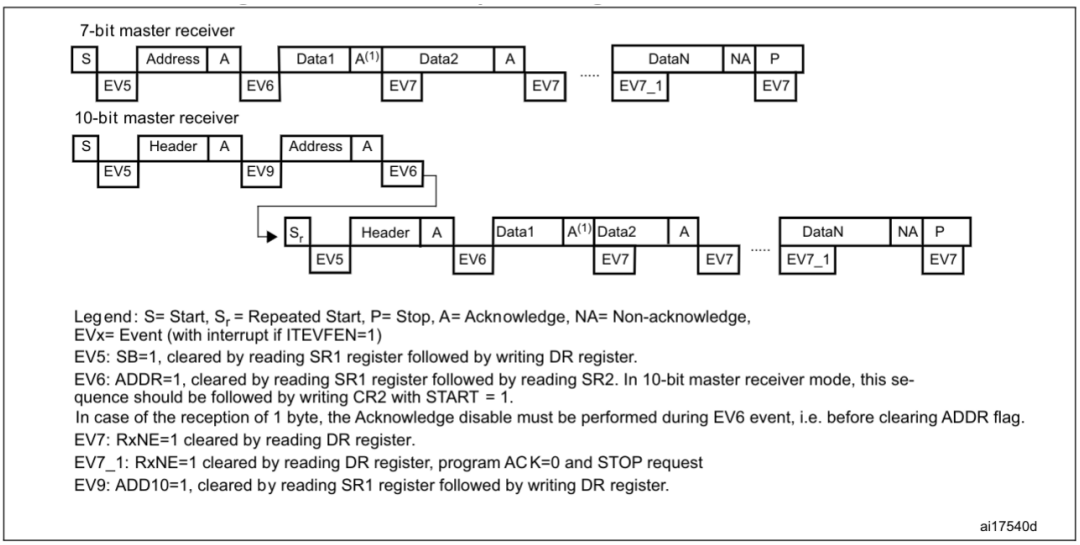
STM32F405 I2C Controler Sequences

The Figures C.1 and C.2 represent the sequences that the firmware must follow for transmission and reception in master mode respectively. And the The Figures C.3 and C.4 represent the sequences that the firmware must follow for transmission and reception in master mode respectively



1. The EV5, EV6, EV9, EV8_1 and EV8_2 events stretch SCL low until the end of the corresponding software sequence.
2. The EV8 event stretches SCL low if the software sequence is not complete before the end of the next byte transmission.

Figure C.1: Transfer sequence diagram for master transmitter



1. If a single byte is received, it is NA.
2. The EV5, EV6 and EV9 events stretch SCL low until the end of the corresponding software sequence.
3. The EV7 event stretches SCL low if the software sequence is not completed before the end of the next byte reception.
4. The EV7_1 software sequence must be completed before the ACK pulse of the current byte transfer.

Figure C.2: Transfer sequence diagram for master receiver

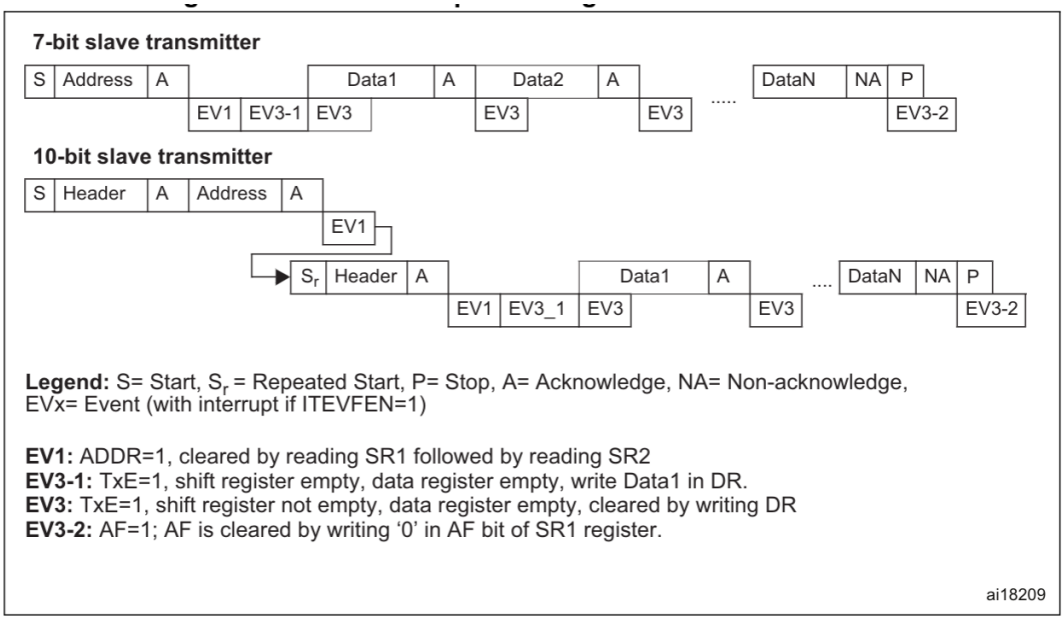


Figure C.3: Transfer sequence diagram for slave transmitter

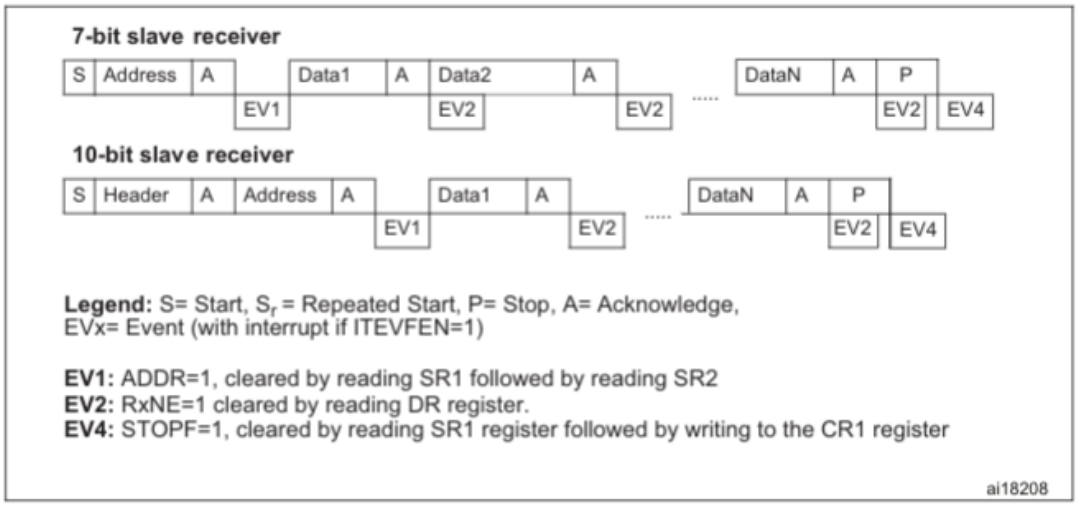


Figure C.4: Transfer sequence diagram for slave receiver

Appendix D

Specifications of the STM32F405 and the LIS3DH

The Table D.1 provides a list of the 8-bit registers embedded in the LIS3DH accelerometer and the corresponding addresses.

Name	Type	Register Address		Default	Comment
		Hex	Binary		
Reserved (do not modify)		00-06			
STATUS_REG_AUX	r	07	0000 0111	—	Output
OUT_ADC1_L	r	08	0000 1000	—	Output
OUT_ADC1_H	r	09	0000 1001	—	Output
OUT_ADC2_L	r	0A	0000 1010	—	Output
OUT_ADC2_H	r	0B	0000 1011	—	Output
OUT_ADC3_L	r	0C	0000 1100	—	Output
OUT_ADC3_H	r	0D	0000 1101	—	Output
Reserved (do not modify)		0E			
WHO_AM_I	r	0F	000 1111	00110011	Dummy register
Reserved (do not modify)		10 — 1D			
CTRL_REG0	rw	1E	0001 1110	00010000	—
TEMP_CFG_REG	rw	1F	0001 1111	00000000	—
CTRL_REG1	rw	20	0010 0000	00000111	—
CTRL_REG2	rw	21	0010 0001	00000000	—
CTRL_REG3	rw	22	0010 0010	00000000	—
CTRL_REG4	rw	23	0010 0011	00000000	—
CTRL_REG5	rw	24	0010 0100	00000000	—
CTRL_REG6	rw	25	0010 0101	00000000	—
REFERENCE	rw	26	0010 0110	00000000	—
STATUS_REG	r	27	0010 0111	—	Output
OUT_X_L	r	28	0010 1000	—	Output
OUT_X_H	r	29	0010 1001	—	Output
OUT_Y_L	r	2A	0010 1010	—	Output
OUT_Y_H	r	2B	0010 1011	—	Output
OUT_Z_L	r	2C	0010 1100	—	Output
OUT_Z_H	r	2D	0010 1101	—	Output
FIFO_CTRL_REG	rw	2E	0010 1110	00000000	
FIFO_SRC_REG	r	2F	0010 1111	—	Output
INT1_CFG	rw	30	0011 0000	00000000	—
INT1_SRC	r	31	0011 0001	—	—
INT1_THS	rw	32	0011 0010	00000000	—
INT1_DURATION	rw	33	0011 0011	00000000	—
INT2_CFG	rw	34	0011 0100	00000000	—
INT2_SRC	r	35	0011 0101	—	—
INT2_THS	rw	36	0011 0110	00000000	—
INT2_DURATION	rw	37	0011 0111	00000000	—
CLICK_CFG	rw	38	0011 1000	00000000	—
CLICK_SRC	r	39	0011 1001	—	—
CLICK_THS	rw	3A	0011 1010	00000000	—
TIME_LIMIT	rw	3B	0011 1011	00000000	—
TIME_LATENCY	rw	3C	0011 1100	00000000	—
TIME_WINDOW	rw	3D	0011 1101	00000000	—
ACT_THS	rw	3E	0011 1110	00000000	—
ACT_DUR	rw	3F	0011 1111	00000000	—

Table D.1: LIS3DH Registers

The Figure D.1 represents the complete mapping of the configuration bits of the I^2C peripheral in the STM32F405.

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	I2C_CR1	Reserved																SWRST	Reserved	ALERT	PEC	POS	ACK	STOP	START	NOSTRETCH	ENGC	ENPEC	ENARP	SMBTYPE	Reserved	SMBUS	PE
	Reset value																	0		0	0	0	0	0	0	0	0	0	0	0		0	0
0x04	I2C_CR2	Reserved																			LAST	DMAEN	ITBUFEN	ITEVTEN	ITERREN	Reserved	FREQ[5:0]						
	Reset value																				0	0	0	0	0		0	0	0	0	0	0	0
0x08	I2C_OAR1	Reserved																ADDMODE	Reserved					ADD[9:8]		ADD[7:1]							ADD0
	Reset value																	0						0	0	0	0	0	0	0	0	0	0
0x0C	I2C_OAR2	Reserved																								ADD2[7:1]							ENDUAL
	Reset value																									0	0	0	0	0	0	0	0
0x10	I2C_DR	Reserved																								DR[7:0]							
	Reset value																									0	0	0	0	0	0	0	0
0x14	I2C_SR1	Reserved																SMBALERT	TIMEOUT	Reserved	PECERR	OVR	AF	ARLO	BERR	TxE	RxNE	Reserved	STOPF	ADD10	BTF	ADDR	SB
	Reset value																	0	0		0	0	0	0	0	0	0		0	0	0	0	0
0x18	I2C_SR2	Reserved																PEC[7:0]								DUALF	SMBHOST	SMBDEFAUL	GENCALL	Reserved	TRA	BUSY	MSL
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0		0	0	0
0x1C	I2C_CCR	Reserved																F/S	DUTY	Reserved	CCR[11:0]												
	Reset value																	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0
0x20	I2C_TRISE	Reserved																										TRISE[5:0]					
	Reset value																											0	0	0	0	1	0
0x24	I2C_FLTR	Reserved																											ANOFF	DNF[3:0]			
	Reset value																												0	0	0	0	0

Figure D.1: stm32f405 i2c memory map

Appendix E

QEMU C Data Structures

E.1 QOM API Core Elements

ObjectClass: Analogous to a class in any OOP language, the C `ObjectClass` structure represents the metadata that describe a type and that is shared by all instances of that type. It holds [44]:

code E.1: `ObjectClass`

```
struct ObjectClass
{
    /* private: */
    Type type;
    GSList *interfaces;

    const char *object_cast_cache[OBJECT_CLASS_CAST_CACHE];
    const char *class_cast_cache[OBJECT_CLASS_CAST_CACHE];

    ObjectUnparent *unparent;

    GHashTable *properties;
};
```

- **Type information**: Unique identifier for runtime type checking
- **Virtual method pointers**: Functions that can be overridden by derived types
- **Interface implementations**: Lists of interfaces this type provides.
- **Shared class properties**: Default values and metadata for all instances

Object: Every object in QEMU inherits from the C `Object` structure, which is the fundamental component of the entire type system. Each individual object is an **object**

instance with its own state, which can be instantiated, configured, introspected, and destroyed. Each `Object` instance encapsulates:

code E.2: `Object`

```
struct Object
{
    /* private: */
    ObjectClass *class;
    ObjectFree *free;
    GHashTable *properties;
    uint32_t ref;
    Object *parent;
};
```

- **Type identification** via a class pointer
- **Reference counting** for memory management
- **Dynamic properties** storage and access
- **Composition tree relationships** (parent pointers) enabling hierarchical organization

TypeInfo: The C `TypeInfo` structure is the complete blueprint for registering a new QOM type at QEMU startup. This structure tells QEMU exactly how to create, initialize, and manage instances of the QOM type.

code E.3: `TypeInfo`

```
struct TypeInfo
{
    const char *name;
    const char *parent;

    size_t instance_size;
    size_t instance_align;
    void (*instance_init)(Object *obj);
    void (*instance_post_init)(Object *obj);
    void (*instance_finalize)(Object *obj);

    bool abstract;
    size_t class_size;

    void (*class_init)(ObjectClass *klass, void *data);
    void (*class_base_init)(ObjectClass *klass, void *data);
    void *class_data;

    InterfaceInfo *interfaces;
};
```

- Type name and parent class (e.g., parent=`"sysbus-device"`)
- Instance and class sizes (memory layout)
- Constructor (`instance_init`), post-constructor (`instance_post_init`), and destructor (`instance_finalize`) callbacks
- Class initialization callback (`class_init`) where virtual methods are assigned

To exemplify the use of the `TypeInfo` structure: The `STM32I2CClass` is registered via `TypeInfo` with parent=`'sysbus-device'`, `instance_size=sizeof(STM32I2CState)`, and `class_init =stm32_i2c_class_init`. This registration enables QEMU to instantiate the I^2C controller and configure its base address, interrupt lines, and master/slave modes at runtime.

ObjectProperty: Each object's configurable properties—e.g., base addresses, enable bits, interrupt vectors—are registered via `ObjectProperty` descriptors. These define getters/setters, type metadata, and default values, enabling firmware configuration without recompilation.

E.2 QEMU Memory

code E.4: `MemoryRegion`

```
struct MemoryRegion {
    Object parent_obj;

    /* private: */

    /* The following fields should fit in a cache line */
    bool romd_mode;
    bool ram;
    bool subpage;
    bool readonly; /* For RAM regions */
    bool nonvolatile;
    bool rom_device;
    bool flush_coalesced_mmio;
    bool unmergeable;
    uint8_t dirty_log_mask;
    bool is_iommu;
    RAMBlock *ram_block;
    Object *owner;
    /* owner as TYPE_DEVICE. Used for re-entrancy checks in MR
       access hotpath */
    DeviceState *dev;

    const MemoryRegionOps *ops;
    void *opaque;
    MemoryRegion *container;
    int mapped_via_alias; /* Mapped via an alias, container might
       be NULL */
    Int128 size;
    hwaddr addr;
    void (*destructor)(MemoryRegion *mr);
    uint64_t align;
    bool terminates;
    bool ram_device;
    bool enabled;
    bool warning_printed; /* For reservations */
    uint8_t vga_logging_count;
    MemoryRegion *alias;
    hwaddr alias_offset;
    int32_t priority;
    QTAILQ_HEAD(, MemoryRegion) subregions;
    QTAILQ_ENTRY(MemoryRegion) subregions_link;
    QTAILQ_HEAD(, CoalescedMemoryRange) coalesced;
    const char *name;
    unsigned ioeventfd_nb;
    MemoryRegionIoeventfd *ioeventfds;
    RamDiscardManager *rdm; /* Only for RAM */

    /* For devices designed to perform re-entrant IO into their own
        IO MRs */
```

```
    bool disable_reentrancy_guard;
};
```

Appendix F

Implementation Code

F.1 STM32F4xx I^{2}C Controler Code

code F.1: stm32f4xx_i2c_config_t

```
/* stm32f4xx I2C parameters used in the internal logic */
typedef struct stm32f4xx_i2c_config_s
{
    uint8_t addr;
    stm32f4xx_opt_t ops;
    bool flg_sb;
    bool flg_start;
    bool flg_stop;
    bool flg_swrst;
    bool flg_addr;
}stm32f4xx_i2c_config_t;
```

code F.2: stm32f4xx_i2c_fsm_trans_t

```
/* stm32f4xx I2C fsm transition */
typedef struct stm32f4xx_i2c_fsm_trans_s
{
        /* State of origin  */
        stm32f4xx_i2c_fsm_state_t           org_st;
        /* Transition Condition */
        stm32f4xx_i2c_fsm_condition_func_t  condition;
        /* Destination state */
        stm32f4xx_i2c_fsm_state_t           dst_st;
        /* Output generated by the transition */
        stm32f4xx_i2c_fsm_output_func_t     output;
}stm32f4xx_i2c_fsm_trans_t;
```

code F.3: `stm32f4xx_i2c_fsm_t`

```
/* stm32f4xx I2C fsm structure */
typedef struct stm32f4xx_i2c_fsm_s
{
        int act_st;     // State where the fsm is
        stm32f4xx_i2c_fsm_trans_t *trans_table; // Transition table
}stm32f4xx_i2c_fsm_t;
```

code F.4: MMIO write dispatcher

```
static void stm32f4xx_i2c_write(void *opaque, hwaddr addr, uint64_t
    value, unsigned size) {
    STM32F4XXI2CState *s = opaque;

    s->config.ops = ops_w;      // Record operation type
    s->config.addr = addr;      // Record register offset

    switch (addr) {
        case STM32F4_I2C_CR1_ADDR:  stm32f4xx_i2c_write_cr1(s,
            value); break;
        case STM32F4_I2C_CR2_ADDR:  stm32f4xx_i2c_write_cr2(s,
            value); break;
        case STM32F4_I2C_DR_ADDR:   stm32f4xx_i2c_write_dr(s, value
            ); break;
        // Other registers updated directly
    }
}
```

code F.5: MMIO read dispatcher

```
static uint64_t stm32f4xx_i2c_read(void *opaque, hwaddr addr,
    unsigned size)
{
    STM32F4XXI2CState *s = opaque;
    uint64_t result = 0x00UL;

    s->config.ops = ops_r;
    s->config.addr = addr;

    if ( s->config.flg_addr && addr != STM32F4_I2C_SR2_ADDR)
        s->config.flg_addr = false;

    if ( s->config.flg_sb )
        s->config.flg_sb = false;

    switch (addr)
    {
        case STM32F4_I2C_CR1_ADDR:   result = s->i2c_cr1;
            break;
        case STM32F4_I2C_CR2_ADDR:   result = s->i2c_cr2;
            break;
        case STM32F4_I2C_OAR1_ADDR:  result = s->i2c_oar1;
            break;
        case STM32F4_I2C_OAR2_ADDR:  result = s->i2c_oar2;
            break;
        case STM32F4_I2C_DR_ADDR:    result = stm32f4xx_i2c_read_dr
            (s);  break;
        case STM32F4_I2C_SR1_ADDR:   result =
            stm32f4xx_i2c_read_sr1(s); break;
        case STM32F4_I2C_SR2_ADDR:   result =
            stm32f4xx_i2c_read_sr2(s); break;
        case STM32F4_I2C_CCR_ADDR:   result = s->i2c_ccr;
            break;
        case STM32F4_I2C_TRISE_ADDR: result = s->i2c_trise;
            break;
        case STM32F4_I2C_FLTR_ADDR:  result = s->i2c_fltr;
            break;

        default:
            qemu_log_mask(LOG_GUEST_ERROR, "STM32␣I2C:␣Bad␣read␣
                offset␣0x%lx\n", addr);
    }

    STM32_DEBUG("Value␣readed:␣0x%02x", (uint32_t)result);

    s->config.ops = ops_na;
    s->config.addr = 0xFF;

    return result;
}
```

F.2 LIS3DH Code

code F.6: LIS3DH device state structure

```
typedef struct LIS3DHState
{
    /* Parent object */
    I2CSlave i2c;

    lis3dh_config_t *config;    /**< Operating mode, scale, ODR */
    lis3dh_params_t *params;    /**< Sensitivity, range, shifts */
    lis3dh_i2c_params_t i2c;    /**< Address pointer, auto-inc  */

    /* GPIO outputs */
    qemu_irq int1;               /**< INT1 output line */
    qemu_irq int2;               /**< INT2 output line */

    /* Registers */
    uint8_t status_reg_aux; // Status Register
    uint8_t adc_1_l;    // 1-Axis Acceleration Data Low Register
    uint8_t adc_1_h;    // 1-Axis Acceleration Data High Register
    uint8_t adc_2_l;    // 2-Axis Acceleration Data Low Register
    uint8_t adc_2_h;    // 2-Axis Acceleration Data High Register
    uint8_t adc_3_l;    // 3-Axis Acceleration Data Low Register
    uint8_t adc_3_h;    // 3-Axis Acceleration Data High Register
    :
    :
    :
    :
    uint8_t act_ths;
    uint8_t act_dur;

} LIS3DHState;
```

code F.7: LIS3DH I2C slave event handler

```
static int lis3dh_i2c_event(I2CSlave *i2c, enum i2c_event event)
{
    LIS3DHState *lis3dh = LIS3DH(i2c);

    switch (event)
    {
        case I2C_START_SEND:    // Start of write operation
            /* Master is starting a WRITE operation
            (sending data to the device) */
            lis3dh->i2c_params.ptr              = 0xFF;
                        lis3dh->i2c_params.auto_increment  = false;
                        lis3dh->i2c_params.address_phase   = true;
            break;

        case I2C_START_RECV:    // Start of read operation
            /* Master is starting a READ operation
            (requesting data from the device) */
                        if (lis3dh->i2c_params.ptr == 0xFF)
                                lis3dh->i2c_params.ptr =
                                   LIS3DH_ADDR_WHO_AM_I;

                        lis3dh->i2c_params.address_phase = false;
            break;

        case I2C_FINISH:        // Stop condition
            /* Master ends the transaction
            (STOP condition) */
            break;

        case I2C_NACK:          // NACK received
            /* Master didn't acknowledge the data */
            break;

        default:                // Should never happen
            return -1;
    }

    return 0;
}
```

code F.8: LIS3DH I2C slave recv (write from master)

```
static uint8_t lis3dh_i2c_recv(I2CSlave *i2c)
{
    LIS3DHState *lis3dh = LIS3DH(i2c);

    uint8_t value = lis3dh_read_register( lis3dh );

    if (lis3dh->i2c_params.auto_increment)
    {
        lis3dh->i2c_params.ptr++;
    }

        return value;
}
```

code F.9: LIS3DH I2C slave send (read by master)

```
static int lis3dh_i2c_send(I2CSlave *i2c, uint8_t data)
{
    LIS3DHState *lis3dh = LIS3DH(i2c);

        if (lis3dh->i2c_params.ptr == 0xFF && lis3dh->i2c_params.
            address_phase)
        {
                lis3dh->i2c_params.ptr = data & LIS3DH_SUB_REG_MASK
                    ;
                lis3dh->i2c_params.auto_increment =  (data &
                    LIS3DH_SUB_AUTO_INC_MASK) != 0x00 ? true : false
                    ;
                lis3dh->i2c_params.address_phase = false;
        }else
        {
                lis3dh_write_register( lis3dh, lis3dh->i2c_params.
                    ptr, data);
                if (lis3dh->i2c_params.auto_increment)
                        lis3dh->i2c_params.ptr++;
        }
        return 0;
}
```

code F.10: LIS3DH QOM properties

```
static void lis3dh_initfn(Object *obj)
{
    object_property_add(obj, "accel-x", "int",
                        lis3dh_get_accel_x,
                        lis3dh_set_accel_x, NULL, NULL);
    object_property_add(obj, "accel-y", "int",
                        lis3dh_get_accel_y,
                        lis3dh_set_accel_y, NULL, NULL);
    object_property_add(obj, "accel-z", "int",
                        lis3dh_get_accel_z,
                        lis3dh_set_accel_z, NULL, NULL);
    object_property_add(obj, "temp", "int",
                        lis3dh_get_temp,
                        lis3dh_set_temp, NULL, NULL);
}
```

Bibliography

[1] T. Instruments, *A Basic Guide to I2C*, Texas Instruments Incorporated, 2022. [Online]. Available: https://www.ti.com/lit/an/sbaa565/sbaa565.pdf

[2] P. Semiconductors, *The I^2C-bus specification*, Philips Semiconductors, 2000. [Online]. Available: https://www.csd.uoc.gr/~hy428/reading/i2c_spec.pdf

[3] A. Ralston, E. Reilly, and D. Hemmendinger, *Encyclopedia of Computer Science*. Wiley, 2003. [Online]. Available: https://archive.org/details/EncyclopediaOfComputerScience/page/n555/mode/2up

[4] Global Market Insights, "Embedded system market analysis," https://www.gminsights.com/industry-analysis/embedded-system-market, 2026.

[5] Embedded.com, "Embedded market study 2023," https://www.embedded.com/wp-content/uploads/2023/05/Embedded-Market-Study-For-Webinar-Recording-April-2023.pdf, 2023.

[6] CodeThink, "Qemu for testing embedded linux," https://www.codethink.co.uk/articles/2024/qemu-testing-embedded-linux/, 2024.

[7] R. R. Cárdenas and J. J. G. Valverde, "Microlab - laboratorio virtual de microcontroladores para la innovación educativa en asignaturas de sistemas electrónicos," https://innovacioneducativa.upm.es/proyectos-ie/informacion?anyo=2024-2025&id=1633, Universidad Politécnica de Madrid, 2025.

[8] J. J. G. Valverde and J. P. Ortiz, "Sivalse - diseño e implementación de herramientas de simulación y validación automática para facilitar el proceso de aprendizaje en asignaturas de sistemas electrónicos," https://innovacioneducativa.upm.es/proyectos-ie/informacion?anyo=2023-2024&id=1224, Universidad Politécnica de Madrid, 2024, código IE24.0904. Accessed: January 24, 2026.

[9] University of California Berkeley, "Berkeley risc," https://en.wikipedia.org/wiki/Berkeley_RISC.

[10] M. Levy, "The history of the arm architecture: From inception to ipo," *ARM IQ*, vol. 4, no. 1, 2005.

[11] S. P. Dandamudi, *ARM Architecture*. New York, NY: Springer New York, 2005.

[12] Arm Holdings, "Arm architecture family," https://en.wikipedia.org/wiki/ARM_architecture_family.

[13] Yorifang, "Armv8 virtualization overview," https://www.openeuler.org/en/blog/yorifang/2020-10-24-arm-virtualization-overview.html, October 2020.

[14] Arm Holdings, "The arm ecosystem ships a record 6.7 billion arm-based chips in a single quarter," https://newsroom.arm.com/news/the-arm-ecosystem-ships-a-record-6-7-billion-arm-based-chips-in-a-single-quarter, February 2021.

[15] STMicroelectronics, "Stm32cubeide: All-in-one development tool for stm32," https://www.st.com/en/development-tools/stm32cubeide.html, 2026.

[16] ——, "Stm32cubemonitor: Real-time application monitoring," https://www.st.com/en/development-tools/stm32cubemonitor.html, 2026.

[17] Arm Developer, "Gnu toolchain downloads," https://developer.arm.com/Tools%20and%20Software/GNU%20Toolchain, 2026.

[18] Arm Holdings, "Arm gnu toolchain documentation," https://documentation-service.arm.com/static/5ea8075a9931941038df1413?token=, 2026.

[19] Fedora Project, "Fedora linux - the fedora project," https://fedoraproject.org/es/, 2026.

[20] Red Hat, "Announcing fedora 42," https://www.redhat.com/en/blog/announcing-fedora-42, 2026.

[21] Microsoft Corporation, "Visual studio code documentation," https://code.visualstudio.com/docs, 2024.

[22] STMicroelectronics, "Nucleo-f411re: Stm32 nucleo-64 development board," https://www.st.com/en/evaluation-tools/nucleo-f411re.html, 2026, accessed: February 1, 2026.

[23] ——, "Nucleo-f446re: Stm32 nucleo-64 development board," https://www.st.com/en/evaluation-tools/nucleo-f446re.html, 2026, accessed: February 1, 2026.

[24] ——, "32f411ediscovery: Stm32f4 discovery kit," https://www.st.com/en/evaluation-tools/32f411ediscovery.html, 2026, accessed: February 1, 2026.

[25] ——, *STM32F411xCE Advanced ARM-based 32-bit MCUs Reference Manual*, https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf, STMicroelectronics, 2023.

[26] ——, *STM32F446xx Advanced ARM-based 32-bit MCUs Reference Manual*, https://www.st.com/resource/en/reference_manual/rm0390-stm32f446xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf, STMicroelectronics, 2023.

[27] ——, *STM32F405xx/STM32F407xx Datasheet*, STMicroelectronics, 2026. [Online]. Available: https://www.st.com/resource/en/datasheet/dm00037051.pdf

[28] ——, *RM0090 Reference Manual: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 Advanced Arm-based 32-bit MCUs*, rev. 19 ed., https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mc pdf, STMicroelectronics, 2021.

[29] S. Studio, "Difference between arduino sensor kit & grove beginner kit," 2024, accessed: February 28, 2026. [Online]. Available: https://files.seeedstudio.com/products/103030375/Difference%20between%20Arduino%20Sensor%20Kit%20%26%20Grove%20Beginner%20Kit.pdf

[30] STMicroelectronics, *LIS3DH: MEMS digital output motion sensor: ultra-low-power high-performance 3-axis "nano" accelerometer*, STMicroelectronics, 2016, accessed: January 04, 2026. [Online]. Available: https://www.st.com/resource/en/datasheet/lis3dh.pdf

[31] ——, *Description of STM32F4 HAL and Low-Layer Drivers*, rev 8 ed., STMicroelectronics, March 2023, accessed: February 28, 2026. [Online]. Available: https://www.st.com/resource/en/user_manual/um1725-description-of-stm32f4-hal-and-lowlayer-drivers-stmicroelectronics.pdf

[32] QEMU Project, "About qemu," https://qemu-project.gitlab.io/qemu/about/index.html.

[33] Wikipedia Contributors, "Fabrice bellard," https://en.wikipedia.org/wiki/Fabrice_Bellard.

[34] QEMU Project, "Tcg ops developer reference," https://gitlab.com/qemu-project/qemu/-/blob/master/docs/devel/tcg-ops.rst.

[35] ——, "Tcg direct block chaining," https://qemu-project.gitlab.io/qemu/devel/tcg.html#direct-block-chaining.

[36] V. Chipounov and G. Candea, "Dynamically translating x86 to llvm using qemu," https://infoscience.epfl.ch/server/api/core/bitstreams/112e0eb6-d151-46b7-ad9d-3db87be5bf45/content, 2010.

[37] QEMU Project, "Tcg frontend operations," https://wiki.qemu.org/Documentation/TCG/frontend-ops.

[38] ——, "Tcg backend operations," https://wiki.qemu.org/Documentation/TCG/backend-ops.

[39] ——, "Qemu internals documentation," https://qemu-project.gitlab.io/qemu/devel/index-internals.html, 2026, accessed: February 1, 2026.

[40] ——, "Qemu api documentation," https://qemu-project.gitlab.io/qemu/devel/index-api.html, 2026, accessed: February 1, 2026.

[41] F. Bellard, "Qemu, a fast and portable dynamic translator," in *Proceedings of the USENIX Annual Technical Conference*, 2005, pp. 41–46.

[42] QEMU Project, "Qemu's object model," https://qemu-project.gitlab.io/qemu/devel/qom.html#qom.

[43] ——, "Mmio operations," https://qemu-project.gitlab.io/qemu/devel/memory.html#mmio-operations.

[44] ——, "Qemu's object model," https://qemu-project.gitlab.io/qemu/devel/qom-api.html#c.ObjectClass.